UNIVERSIDAD DE SANTIAGO DE COMPOSTELA

ESCUELA TÉCNICA SUPERIOR DE INGENIERÍA

Desarrollo y diseño de un videojuego RPG por turnos

Autor:
Abraham Rial Carreira

Tutor:
José Varela Pet

Grado en Ingeniería Informática

Julio 2020

Trabajo de Fin de Grado presentado en la Escuela Técnica Superior de Ingeniería de la Universidad de Santiago de Compostela para la obtención del Grado en Ingeniería Informática

Agradecimientos

A mi tutor, José Varela Pet, por guiarme en el desarrollo y documentación del proyecto. A mi familia, por su apoyo y ánimo a lo largo del grado.

Resumen

Este proyecto, llamado Fantasia: Un reino perdido, es un proyecto inicialmente concebido en 2013, es decir, antes de iniciar los estudios en ingeniería informática, y sin conocimientos de programación, que consiste en el desarrollo de un videojuego, en concreto un TRPG (Tactical Role Playing Game), de estrategia por turnos, inspirado en videojuegos como Tactics Ogre, Final Fantasy Tactics o Pokemon Conquest.

Este TFG consistirá contempla el desarrollo de un prototipo jugable del videojuego, que demuestre un funcionamiento adecuado del motor en el que se basa. El objetivo es que la estructura básica del videojuego sea completamente funcional, de manera que la jugabilidad esté más o menos completa, dejando de lado aspectos para los que no tenemos tiempo en el alcance de un TFG, y aquellos que no tienen que ver con la carrera, como composición de música, creación de gráficos 2D, o desarrollo de una historia.

En definitiva, la meta del desarrollo es la obtención de un motor donde se puedan ejecutar las tareas de planificación táctica en un mundo separado por regiones, personalización individual de unidades, y combate táctico en mapas separados en cuadrículas.

Índice general

1. Introducción **1**
1.1. Contextualización del proyecto 1
1.1.1. Inspiración del proyecto 1
1.1.2. Justificación del proyecto 2
1.1.3. Objetivos generales del proyecto 2
1.1.4. Análisis del género 3
1.1.5. Análisis de mercado 5
1.2. Descripción y alcance del proyecto 6
1.2.1. Descripción del juego 6
1.2.2. Mecánicas del juego 7
1.2.3. Alcance del proyecto 11
1.2.4. Tecnologías utilizadas 12

2. Planificación y presupuestos **15**
2.1. Gestión de configuración 15
2.1.1. Infraestructura del proyecto 15
2.1.2. Control de cambios 17
2.2. Gestión de riesgos 18
2.2.1. Clasificación de los factores de riesgo 18
2.3. Clasificación de los riesgos 20
2.3.1. Planes de respuesta 20
2.3.2. Datos de clasificación de riesgos 20
2.3.3. Tablas de riesgos iniciales 23
2.3.4. Tablas de riesgos progresivas 29
2.4. Gestión temporal 31
2.4.1. Cronograma 32
2.4.2. Hitos del proyecto 32
2.4.3. Evolución del cronograma 33
2.5. Gestión de costes 33
2.5.1. Costes de recursos humanos 33
2.5.2. Costes materiales 34
2.5.3. Costes indirectos 34
2.5.4. Costes totales 34

2.5.5. Coste alternativo 34
2.6. Gestión de comunicaciones 35

3. Especificación de requisitos **37**
3.1. Requisitos del proyecto 37
3.1.1. Requisitos funcionales 37
3.1.2. Requisitos no funcionales 42
3.1.3. Requisitos de información 43
3.1.4. Requisitos de hardware 45
3.2. Actores 46
3.3. Casos de uso 46

4. Diseño **59**
4.1. Modelo de información 59
4.2. Módulos del proyecto. 60
4.2.1. Módulos 60
4.2.2. Relaciones intermodulares 64
4.3. Diseño de bajo nivel 65
4.3.1. Estructuras de datos. 65
4.3.2. Funciones de gráficos 69
4.3.3. Pila de estados 74

5. Validación **79**
5.1. Pruebas del proyecto 79
5.1.1. Pruebas de casos de uso 80
5.1.2. Pruebas de límites 81
5.2. Análisis de requisitos implementados 81

6. Conclusiones y posibles ampliaciones **83**
6.1. Conclusiones 83
6.2. Posibles ampliaciones 83

A. Manuales técnicos **85**

B. Manual de usuario **87**
B.1. Instalación 87
B.2. Utilización del juego 87

Bibliografía **89**

Índice de figuras

2.1. Estructura de carpetas 16
2.2. Cronograma inicial 32
2.3. Nuevo Cronograma 33

3.1. Casos de uso 48

4.1. Modelo de información 61
4.2. Creación de personaje 66
4.3. Definición de clase 67
4.4. Fragmento de función de ataque 68
4.5. Fragmento de función de cura 68
4.6. Importación de mapa Tiled 69
4.7. Código de cuadrícula de mapa 70
4.8. Mapa importado [7] 70
4.9. Animación de movimiento 72
4.10. Pantalla de inicio 72
4.11. Estado de FadeIn 74
4.12. Codigo de la pila de estados 76
4.13. Codigo del estado del menú de combate 77

Índice de cuadros

Capítulo 1

Introducción

1.1. Contextualización del proyecto

1.1.1. Inspiración del proyecto

Este proyecto está inspirado en videojuegos del género TRPG, que se ve definido por tres rasgos principales:

Control de un grupo de unidades, el cual se puede personalizar en distintos grados, desde decidir un único objeto que pueden llevar para utilizar en combate, a tener un complejo sistema donde se puede cambiar el equipamiento, técnicas, clases e incluso raza del personaje. Este suele estar acompañado de un sistema de progresión, donde el combate o entrenamiento potencia a las unidades y les proporciona opciones estratégicas.

Combate táctico por turnos en mapas divididos en casillas, generalmente cuadradas o hexagonales, que representan distancias, y determinan movimiento, alcances, y a veces visión. Además, determinaban distintas características del terreno, como niveles de altura, obstáculos como nieve o trampas, y propiedades como protección o cobertura.

Gestión de recursos, que puede ir desde cosas simples, como la gestión de equipamiento, habilidades o consumibles de las unidades, hasta cosas más complejas, como la gestión de las unidades a lo largo de un mapamundi. Este último es típico de los simuladores de guerra/conquista, de los cuales nuestro proyecto también toma inspiración.

Aunque no es un rasgo necesario, estos videojuegos suelen además tener en su inmensa mayoría una temática fantástica, inspirada en mitología generalmente europea o asiática, con raros casos tomando prestado de la ciencia ficción.

Algunos de los primeros ejemplos de videojuegos TRPG definidos por estos rasgos son Tactics Ogre o Final Fantasy Tactics, ambos juegos de fantasía medieval con magia, que siguen un protagonista líder de su grupo (clan, ejército, tribu, etc), en busca de un artefacto que suele ser central al universo en el que se presenta el juego.

En ejemplos más recientes y modernos del genero se encuentra el muy famoso X-COM, donde sigues a un escuadrón militar secreto que busca defender la tierra de una invasión alienígena en combates tácticos, o Fell Seal: Arbiter's Mark, una de la inspiraciones principales del proyecto, pues es un ejemplo de proyecto personal que se desarrolló hasta el éxito comercial con un equipo inicial de solo 2 personas, y que a pesar de ser reciente, sigue los patrones de los padres del género, incluso utilizando gráficos basados en sprites como homenaje a los mismos.

1.1.2. Justificación del proyecto

Nuestro proyecto de desarrollo de un videojuego no entra en el molde clásico de un TFG de ingeniería informática, ya que mientras estos suelen constar de un programa o módulo de un programa con un objetivo específico que se centra en la funcionalidad principal del mismo, este TFG busca desarrollar un proyecto personal en el cual hay mucha mayor libertad de desarrollo, centrándose en obtener la mejor versión posible del programa dentro de un contexto de TFG, lo que permite adaptarse a medida que se desarrolla el proyecto. Se nota también que la temática del proyecto, el desarrollo de un videojuego, es algo inusual para el grado, y la razón de escoger esta temática es la siguiente.

En este punto, es apropiado señalar que el interés en el proyecto del desarrollador tiene un origen que coindice con su decisión de iniciar los estudios de Ingeniería Informática. Los videojuegos han sido siempre una de sus fuentes principales de entretenimiento, y estos fueron los que inicialmente encendieron su interés por la informática, y eventualmente la programación, y son la razón de que buscase ampliar mi conocimiento de los mismos con esta carrera. El objetivo del desarrollador en el mundo laboral es poder integrarse como desarrollador y/o diseñador de videojuegos, por lo que el TFG ofrece una oportunidad de aplicar y demostrar las habilidades y conocimientos obtenidos a lo largo de estos años.

1.1.3. Objetivos generales del proyecto

Aunque nuestro objetivo exacto no es inmediatamente notable, debido a que la aplicación que desarrollamos no se centra alrededor de una única función principal, si está bien definido, y lo que podrá cambiar a lo largo del desarrollo son los requisitos de importancia media y baja, que podemos adaptar si tenemos nuevas

ideas para el juego, encontramos dificultades para algunos de ellos, etc.

Nuestro objetivo de programación para el videojuego es obtener un prototipo funcional del mismo en el que se integre principalmente los 3 rasgos del género que hemos mencionado, es decir, debemos tener un grupo de unidades con un sistema de progresión y personalización, que pueda participar en combates tácticos contra otros grupos de unidades controladas por IA, y como se ha mencionado, serán distribuidas en un mapamundi dividido en regiones, propio de los simuladores de conquista. Además de esto, buscamos poder introducir ciertas mecánicas, que aunque no típicas del genero, consideramos importantes para el prototipo, ya que queremos que sea parte del videojuego final que buscamos desarrollar a partir de este prototipo, y de los que hablaremos más en profundidad en las mecánicas del juego.

El objetivo del documento es el de todo TFG, documentar la planificación, diseño, pruebas y gestión del proceso de desarrollo del programa.

1.1.4. Análisis del género

Para saber cuales son las partes más importantes del proyecto, lo primero que debemos saber es lo que convierte un juego de este género en exitoso, y para ello se revisarán dos de los juegos más exitosos del género, el recientemente desarrollado X-Com2, y un clásico del género, Final Fantasy Tactics A2.

X-Com 2 es un RPG táctico cuyo foco se basa no tanto en lo típico en un RPG, la progresión invidual de un pequeño grupo de personajes de gran importancia en una historia épica, sino en un escuadrón de soldados personalizables, pero reemplazables", donde no hay ninguno más especial que cualquier otro, y que mueve el foco de la historia de los personajes que participan en ella, a las operaciones que realizan y el comandante que los dirige, además de dar mucha más importancia a la gestión de recursos, ya sean humanos, económicos, equipamiento disponible, tecnología, etc. Su éxito se debe en gran parte a 3 ejes:

-La personalización de los personajes, la cual se siente mucho más significativa que en la mayoría de juegos del género debido a que estos pueden ser modificados por el jugador con gran detalle, y si se pierden en combate se pierden permanentemente. La combinación de estos factores facilita la inmersión del jugador en el mundo del juego y hace que tomen cariño a cada personaje, que ha sido entrenado y cuidado por el jugador a lo largo de su tiempo de juego, formando orgánicamente historias en las que las acciones tomadas en el juego cobran vida en la imaginación del jugador.

-La profundidad táctica sin sacrificar la simplicidad del juego. Al contrario que

en muchos otros juegos, donde se pueden encontrar gran cantidad de habilidades o armas cuyas diferencias son mínimas, el sistema de equipamiento y habilidades de X-Com es compacto y sencillo, ofreciendo suficiente variedad como para abrir distintos caminos y estrategias viables, pero sin avasallar al jugador con información que apenás tendrá impacto en el juego. Un ejemplo de esto es el sistema de clases, donde cada una de las clases base solo tiene disponibles hasta 14 habilidades, de las cuales pueden escoger 7 como máximo, pero cuya sinergía entre habilidades y compañeros de equipo permite crear conjuntos de habilidades muy diversos que cambian el ángulo desde el que puedes acercarte a cada misión.

-La personalización de la experiencia de juego. Debido a la gran cantidad de opciones que da el juego, cada jugador puede adaptar su experiencia a su gusto, teniendo un juego calmado y fácil con el que poder disfrutar la historia, o un desafío táctico extremo, donde el más mínimo error puede ser el fin de la partida, y esta personalización se ve además amplificada por la participación de la comunidad de juego en la creación de "Mods", modificaciones del juego hechas por los jugadores con las herramientas proporcionadas por los desarrolladores, que resultaron en proyectos casi tan grandes como el juego original en sí, siendo el más notable el Mod "Long War", el cual llevó a sus creadores a formar un equipo que acabó entrando en la industria como desarrolladores bajo el nombre de "Pavonis Interactive".

Es alrededor de estos 3 ejes que X-Com construyó un juego extremadamente exitoso, que llegó a más de un millón de ventas, y que engloba los pilares del diseño moderno de TRPGs.

Por otro lado tenemos Final Fantasy Tactics A2 (al que a partir de ahora llamaremos FFT), un juego mucho más clásico, tanto en edad, pues la entrega más reciente de la saga salió en 2007, como en diseño, ya que sigue el patrón en el que el jugador se mete en el punto de vista de un protagonista, lider de un clan, en el que, aunque sigue habiendo personajes personalizables, la historia se centra en un pequeño grupo de personajes pertenecientes al mundo del juego, rígidos y poco modificables, siendo los demás un accesorio a la jugabilidad más que a la historia, y es en la jugabilidad en lo que nos centraremos para este análisis.

Hay 2 grandes pilares en los que se apoya FFT:

-La profundidad del sistema de progresión y desarrollo de los personajes. Si hay algo por lo que es conocido FFT, es su sistema de "Trabajos", en el cual distintas clases están bloqueadas detrás de la obtención de habilidades de otras. Un ejemplo de esto es la clase Ninja, que tiene poderosas habilidades, como la capacidad de esgrimir 2 armas al mismo tiempo, pero que no puede obtenerse en un principio, ya que requiere haber aprendido varias habilidades del trabajo "Ladrón."antes de cambiar a esa clase. Este sistema además permite utilizar una clase secundaria,

de manera que cada personaje puede combinar las habilidades de 2 clases para adaptarse a distintas situaciones y obtener poderosas combinaciones, como una habilidad que permite utilizar armas de 2 manos en una sola mano, y la previamente mencionada habilidad de llevar 2 armas al mismo tiempo, un personaje podría equiparse con 2 armas de dos manos para obtener un enorme incremento en su poder de ataque.

-La atmósfera y temática del juego. FFT se coloca en un mundo fantástico, colorido y animado, que ayuda al jugador a seguir la historia sus personajes. Debido a que el juego se centra en un viaje alrededor del mundo de Ivalice (donde se sitúa este), hay muchas oportunidades para mostrar distintos lugares, cada uno con sus atracciones y maravillas, a medida que observamos también el progreso del conflicto y los personajes, y que se combina para formar un escenario en el que el jugador se interese de verdad, y una aventura que se pueda vivir a través del punto de vista de sus personajes.

Para resumir lo que hemos aprendido de esto, los rasgos principales a los que más importancia daremos al diseñar nuestro juego serán la profundidad del sistema de progresión, sin por ello sacrificar simplicidad e intuitividad, y la personalización de personajes, observando que incluso si nuestra historia se acaba centrando en personajes que nosotros diseñemos, deberíamos permitir la personalización de estos mismos hasta cierto punto, teniendo en cuenta el equilibrio entre la libertad del jugador y la cohesividad de los personajes y la historia.

1.1.5. Análisis de mercado

Para finalizar la introducción, se hablará un poco del mercado de videojuegos, para demostrar la viabilidad comercial de nuestro proyecto, y ver que el desarrollo de videojuegos es una opción laboral viable para un estudiante de ingeniería informática.

La industria de videojuegos es muy joven comparada con muchas otras, pues no llega siquiera a los 50 años, siendo Atari la primera consola comercial en 1971, hace 49 años. Desde esa época, esta industria ha experimentado un crecimiento constante, y cada vez más rápido, pasando de un mercado niché, a una sensación global.

En los últimos años, en E.E.U.U., solo el mercado de videojuegos de ordenador, sin incluir consolas ni juegos de móvil, crecieron de 16000 millones de dolares en 2011, a 30000 millones hoy día (https://www.wepc.com/news/video-game-statistics/#video-gaming-industry-overview), y en España la industria de videojuegos ha pasado de 900 millones de euros en 2009, a más de mil millones en 2019 (https://es.statista.com/estadisticas/472618/prevision-de-facturacion-de-la-industria-de-los-videojuegos-en-espana/).

Con el avance de la tecnología, se predice que estos mercados crecerán aún

más, siendo el sector de videojuegos de móvil uno de los más rápidamente crecientes debido a la potencia de los dispositivos actuales, además de su conveniencia; y se tienen grandes expectativas para tecnologías como la realidad virtual en el desarrollo de videojuegos, que podrían traer no solo nuevas formas de ocio, sino experiencias completamente nuevas para sus usuarios.

1.2. Descripción y alcance del proyecto

1.2.1. Descripción del juego

Aunque no va a ser plenamente desarrollada en este TFG debido a que lo que buscamos es un prototipo funcional, la estética, temática e historia del videojuego servirán para guiar parte de las mecánicas y el desarrollo del mismo, por lo que sería adecuado tener al menos una descripción básica de estos, que es lo que se formalizará en este apartado.

La temática principal del juego será de fantasía medieval, lo que se traduce a un mundo falto de tecnología, pero que contiene magia, criaturas mágicas y artefactos mágicos. Estos últimos son notables porque pueden actuar de substituto a ciertas tecnologías, difuminando la barrera entre lo ancestral y lo contemporáneo (por ejemplo, un cristal mágico que brille eternamente podría substituir una lámpara o bombilla modernas).

El escenario del juego es el mundo ficticio de Fantasia, separado en distintas regiones constantemente en guerra, donde cada líder de región (siendo estos de distintos tipos: reyes, líderes de clan, sacerdotes supremos, etc.) posee una parte de un artefacto legendario conocido como la Estrella Guía, el cual es la razón principal de las guerras, ya que todas las regiones desean reunir los trozos o, como mínimo, defender el propio por distintas razones. La razón de que la Estrella sea tan codiciada es que una antigua leyenda dice que cuando se reúnan todos los fragmentos, esta emitirá una luz que abrirá el camino al Reino Perdido", una región perdida en el tiempo en la que se dice que hay incontables riquezas, herramientas creadas con tecnologías olvidadas, y por encima de todo ello, el elixir de Ygg, una poción mágica capaz de dar vida eterna al que la consuma.

El jugador adoptará el rol de uno de los líderes de región, escogido al iniciar el juego (habiendo por lo tanto tantas historias individuales como regiones), y generalmente deberá conquistar las demás regiones para cumplir el objetivo de la facción escogida y descubrir el camino al Reino Perdido.

En este mundo, además de la presencia de criaturas fantásticas como dragones, grifos o los ancestrales elementales, criaturas hechas de energía pura que han

existido desde el inicio de los tiempos. Tendremos también distintas razas, las cuales tendrán sus propias ventajas y desventajas, además de algunas clases exclusivas a las que otras razas no tendrán acceso. Por ejemplo, podríamos tener un típico elfo, que por la raza a la que pertenece obtenga bonificaciones de destreza, y penalizaciones de fuerza, y que pueda obtener la clase exclusiva de Cazador del Bosque, una variación de la clase de Arquero que se centre en la destreza adicional y sincronía con la naturaleza que los elfos suelen tener en escenarios de fantasía.

Finalmente debemos notar que los lugares que el jugador se encontrará en sus viajes estarán desolados debido a los años de guerra constante, sobre todo en las afueras de las distintas regiones, acentuándose más y más a medida que se acercan a las fronteras.

1.2.2. Mecánicas del juego

En este apartado se describirán las mecánicas que se desean implementar en el juego, con la intención de ofrecer una aproximación informal a los requisitos funcionales. Se incluirá información que justifica la elección de estas mecánicas, pues son de gran relevancia para orientarse en el desarrollo y para evaluar si su implementación fue adecuada. Esto hará mucho más simple la obtención de requisitos y sirve como orientación para los lectores de menor nivel técnico.

Las unidades

En esta sección se describen las mecánicas que rigen la personalización, progreso, y funcionamiento general de las unidades como entidades (es decir, sus estadísticas, atributos, etc.). Las acciones que pueden tomar y como se ven afectadas por el campo de batalla se discutirá en la sección de combate.

Tipo

Debido a que es determinante de como funcionan las demás mecánicas, lo primero que se explica es el tipo. Las unidades podrán pertenecer a dos tipos, humanoides y monstruosas:

Unidades humanoides: Las unidades humanoides son los humanos y las razas basadas en estos mismos. En este grupo entran: elfos, enanos, tritones... Las unidades humanoides tienen las siguientes características:

Equipamiento: son capaces de equiparse distintos objetos, y aunque lo específico varíe entre raza y clase, todas las razas humanoides pueden disponer de:

un arma, que afecta a su poder de ataque y determina rango de ataque y habilidades que puede utilizar; un objeto de mano izquierda, que puede ser un escudo o una segunda arma; una armadura, que afectará el poder defensivo de la unidad; y dos accesorios, que tienen efectos específicos en vez de afectar una estadística concreta.

Clases: se rigen por un sistema de clases, que determinan sus habilidades, sus puntos débiles y fuertes, el equipamiento que pueden llevar, o las técnicas que pueden aprender. Para determinar las clases a las que puede acceder una unidad, se tendrán en cuenta sus estadísticas y nivel, su raza, la experiencia que tiene en otras clases, los títulos que haya adquirido, o cualquier combinación de estas (por ejemplo, una clase básica puede requerir solo llegar a cierto nivel, mientras que otra más avanzada, como un "Sabio", podría requerir cierto valor de Ïnteligencia", o haber aprendido cierta cantidad de conjuros en otra clase mágica).

Unidades monstruosas Las unidades monstruosas son criaturas ficticias basadas en animales o criaturas de mitos y leyendas. En este grupo entran: dragones, muertos vivientes, demonios... Las unidades monstruosas tienen las siguientes características:

Rasgos: como substituto del equipamiento, las unidades monstruosas tienen rasgos, elementos distintivos de su raza, que pueden tener desde su creación u obtener mediante ciertas acciones y que cambian sus propiedades permanentemente (por ejemplo, una marca de hielo podría cambiar el tipo de alienta que escupe un dragón).

Evoluciones: se rigen por árboles evolutivos basados en su raza original. Este sistema funciona de manera similar a las clases, pero los cambios son permanentes y se organizan de manera jerárquica, es decir, una unidad debe pasar por ciertas evoluciones antes de tener acceso a otras (por ejemplo, un esqueleto podría evolucionar a vampiro, pero tendría que pasar antes por ser un zombie).

1.2 Mecánicas comunes.

Todas las unidades, independientemente de su tipo, se rigen por el mismo sistema de combate, y por lo tanto tienen las mismas características principales.

Nivel: representa el poder aproximado de una unidad respecto a su potencial máximo. Todas las unidades que el jugador puede controlar tienen un nivel mínimo de 1, y un máximo de 99. Para subir de nivel, una unidad debe realizar acciones que progresen el combate, como curar a aliados, derrotar a enemigos, o interactuar con el terreno, lo que les proporciona experiencia. Exclusivamente, el personaje principal puede adquirir experiencia mediante eventos, como oportunidades de entrenamiento o negociaciones que formen parte de la historia del juego. Cuando una unidad acumula 100 puntos de experiencia, estos se consumen para que suba de nivel, lo cual implica un aumento en sus características básicas. Para evitar que se adquiera experiencia con excesiva facilidad, las acciones que realice

un personaje y que tengan como objetivo unidades de menor nivel que este darán experiencia reducida, mientras que si el objetivo es de mayor nivel otorgarán más experiencia.

Atributos: representan las capacidades físicas de una unidad. A nivel 1 se determinan a partir de la raza y clase con los que se creó a la unidad, y aumentan dependiendo de las que son en el momento en que se sube de nivel. Los atributos son: fuerza, destreza, energía, resistencia y tenacidad.

Estadísticas: representan las capacidades de combate de una unidad. Se calculan a partir del equipamiento, las habilidades y los títulos, y la mayoría se ve afectada por ciertos atributos. Son los siguientes:

Ataque mágico/físico: indica la potencia de los ataques de la unidad. El mágico se ve afectado por la energía de la unidad, además de armas como cetros o varitas. El físico se ve afectado por fuerza o destreza, dependiendo del arma utilizada. El ataque también afecta la potencia de efectos de recuperación, y se usa el mágico para hechizos y el físico para habilidades.

Defensa mágica/física: indica la capacidad para defenderse de ataques de la unidad. La mágica se ve afectada por la resistencia, mientras la física se ve afectada por la tenacidad.

Salud: indica la capacidad para recibir daño de la unidad. Si llega a 0 la unidad es derrotada. Depende mayormente del nivel, clase y raza.

Maná: indica la capacidad de la unidad para lanzar hechizos o utilizar habilidades. Todas las unidades entran en combate sin maná, que se regenera cada turno, hasta un máximo determinado por la unidad.

Títulos: marcas de honor que reciben las unidades por haber realizado ciertas acciones específicas. Son adquiridos automáticamente cuando la unidad cumple los requisitos de los mismos, y pueden afectar a los atributos, las estadísticas, las habilidades de una unidad, e incluso dar acceso a ciertas clases.

Las regiones

Las regiones serán las distintas secciones en las que se separa el mapamundi, en las cuales se podrán estacionar un número variable de unidades. Estas regiones tendrán las siguientes características:

Capacidad máxima: dependiendo del tamaño y otras características de la región, esta tendrá un máximo de unidades, generalmente alrededor de 6, que puedan estar estacionadas al mismo tiempo.

Edificios: en cada región de podrán construir distintos edificios, que puedes proporcionar beneficios tácticos, como centros de entrenamiento, defensivos, como murallas, o económicos, como tiendas.

Adyacencia: cada región estará adyacente a un mínimo de una región, de manera que las unidades estacionadas en una solo puede moverse a una región

aliada o atacar una enemiga si la región objetivo está adyacente a la suya.

Beneficios mensuales: cada región proporcionará un beneficio al final de cada mes que pase en el juego, que puede incluir un beneficio monetario, o materiales u objetos consumibles.

Región original: cada región individual pertenece a un conjunto de regiones, de manera que controlar todas las regiones que pertenecen al mismo conjunto puede otorgar bonificaciones únicas o mejorar las individuales de cada región.

El combate

Cada combate debe realizarse en un mapa específico, con un bando agresor y uno defensor, pudiendo en ciertos eventos existir una facción neutral, y que debe tomar los siguientes datos de otras partes del programa para organizarse:

Ejército agresor/defensor: para iniciar un combate, debemos tomar del mapa de regiones que unidades hay en la región atacada, y cuales se han movido a esta para invadirla, lo cual determinará las unidades a la que cada bando tiene acceso en el combate, además de sus posiciones iniciales en el mapa.

Mapa asignado: el mapa en el que ocurre cada combate depende exclusivamente de la región que ha sido invadida, de manera que tendremos tantos mapas de combate distintos como regiones.

Instalaciones: debemos recoger de la región atacada la lista de estructuras construidas para determinar si debemos alterar el mapa base, como añadir una muralla para el lado defensor si el jugador que controla la región la ha construido.

Eventos actuales: ciertos eventos como desastres naturales, clima o magias de efecto global pueden activar efectos permanentes en el campo de batalla.

Los mapas de combate

Finalmente, los mapas de combate deben tener ciertas características que los distinguen unos de otros, de manera que cada uno de ellos ofrezca distintas posibilidades estratégicas.

Terreno: cada casilla del mapa debe tener asignado un tipo de terreno, que tendrá distintas propiedades, como costes de movimiento, elevación, mejoras defensivas, o efectos especiales.

Tamaño: el tamaño de los mapas se medirá en la cantidad de casillas de cada uno de sus lados, expresado con la tupla (x,y).

Región: cada mapa está asignado a una región, de manera que los combates solo ocurren en dicho mapa si la región está siendo invadida.

1.2.3. Alcance del proyecto

Enunciado del proyecto

La finalidad de este proyecto radica en la creación de una aplicación para el sistema operativo Windows y de su correspondiente manual de uso. La aplicación se alojará en el sistema del usuario a partir de la extracción de un fichero ZIP que deberá contener todos los ficheros necesarios para su correcto funcionamiento, así como la estructura de directorios precisa.

Se valorará la creación de un instalador automático de manera que el anterior proceso se realice de manera automática, pero puede acabar no siendo necesario por ser el objetivo del proyecto la obtención de un prototipo.

El manual de uso deberá explicar cómo usar la aplicación y qué requerimientos de sistema exige.

Alcance del producto a desarrollar

Debido a las limitaciones temporales inherentes a un Trabajo de Fin de Grado, no se implementarán todas las funcionalidades previstas para el proyecto general y se abarcarán una serie de necesidades básicas. Se desarrollarán los módulos correspondientes al combate, a la personalización de unidades, y a la gestión de recursos y ejércitos en mapa táctico. Por lo tanto el jugador podrá mover sus tropas en un mapa táctico de tamaño reducido, y entrar en combates contra ejércitos específicamente programados para la prueba, tras los cuales obtendrá experiencia y podrá personalizar y potenciar sus tropas hasta cierto punto.

En relación al mundo creado, el juego transcurrirá en un sector reducido que incluirá solo 3 regiones. Sin embargo, todo el análisis y desarrollo se realizará con la premisa de que existirán muchas más regiones para facilitar su implementación futura.
Para el apartado gráfico se utilizarán tanto texturas como sprites externos no creados por el desarrollador, con el objetivo de demostrar la capacidad de incluir gráficos personalizados sin tener que consumir más tiempo del necesario en tareas secundarias como la creación de los mismos. Las unidades dispondrán de todas sus características, pero el conjunto de habilidades, clases, razas y otros elementos que se implementará en el prototipo será reducido para poder demostrar su funcionamiento adecuado sin que consuma una cantidad excesiva de tiempo en tareas secundarias.

1.2.4. Tecnologías utilizadas

Aquí se describen las tecnologías que utilizamos para el proyecto, y se justifican las elecciones realizadas.

Hardware

El hardware simplemente se limita al equipo informático del desarrollador, utilizado a lo largo de la carrera, consiste de un portatil ASUS con las siguientes características:

-Sistema Operativo: Windows 8, sistema de 64bits
-Procesador: Intel Core i7-4720HQ, 2.6 GHz
-Memoria Ram: 16GB
-Tarjeta de video: NVIDIA GeForce GTX960M

Documentación

Libre Office: Como editor de documentos de texto preliminar, porque era el programa ya disponible en el equipo informático a usar, pero la razón de que no buscásemos una alternativa es que es perfectamente adecuado para nuestros propósitos, ofreciendo un editor sólido que, además, es software libre de código abierto.

TeXstudio: Teniendo en cuenta que LaTeX es el sistema de texto más adecuado a los textos de contexto científico y tecnológico, la única decisión real era que editor en concreto utilizar. La elección final fue TeXstudio, debido a la familiaridad del desarrollador con el mismo.

StarUML: Se escogió para modelado UML debido a la familiaridad que el desarrollador tiene con el programa, siendo el utilizado para modelado UML a lo largo de la carrera.

Desarrollo

Lua: El lenguaje de programación que elegimos para este proyecto es Lua, un lenguaje de scripting muy popular para la programación de videojuegos. Es un lenguaje ligero e inicialmente sencillo, pero que hace utilización de librerías para expandir mucho sus posibilidades. Es un lenguaje orientado a la programación de objetos, que utiliza tipos dinámicos y basa sus estructuras de datos en tablas. Hubo varias razones para la elección de Lua como lenguaje para el proyecto. El interés inicial vino del primer contacto con el lenguaje en el videojuego DotA2, donde se utiliza para la programación de sus mapas personalizados, que forman parte del llamado .ªrcade". Tras esto, se investigó brevemente sobre el lenguaje, hasta descubrir su utilización en videojuegos conocidos, como Baldur's Gate, The

Witcher, World of Warcraft, y el archiconocido Angry Birds. Su amplia utilización en distintos videojuegos encendió un interés en un proyecto que utilizase dicho lenguaje, que fue cuando se descubrió el framework Love2D, del cual debemos hablar antes de justificar nuestra elección de Lua, ya que van muy unidas.

Love2D: Love2D es un framework para el desarrollo de videojuegos en 2D. Es un framework gratuito y open-source programado en C++ pero que utiliza Lua como lenguaje de programación. En este framework han sido desarrollados videojuegos como Mari0(una versión modificada del clásico Super Mario Bros que da a Mario acceso a una pistola de portales como las vistas en la saga Portal) o Move or Die. Las inspiraciones principales del desarrollador a dedicarse a la programación de videojuegos fueron viejos videojuegos en 2D, basados en sprites, a los cuales Love2D parecía adaptarse a la perfección. El motor Love2D y el lenguaje de programación Lua fueron escogidos principalmente por esta razón para el proyecto, pues esto supone una gran oportunidad de aprendizaje sobre tecnologías que son nuevas para el autor del TFG.

Atom Text Editor: Atom es un editor de código fuente de código abierto con soporte para múltiples plug-ins y control de versiones integrado. La elección de Atom como nuestro editor de texto se debe a la elevada opinión del mismo en los foros de Love2D, debido sobre todo a uno de los plugins que ofrece, love-ide (acompañado del paquete language-lua para la asistencia de código), que nos permite utilizar Atom no solo como un editor de texto sino como un IDE para el lenguaje Lua bajo el framework de Love2D, en el que podemos fácilmente compilar y probar nuestro proyecto con un solo botón.

Tiled Map Editor: Tiled es un editor de mapas 2D con importación directa a Love2D. Esta aplicación es la manera más sencilla de crear mapas de combate basados en cuadrículas, y la importación directa que nos ofrece es también muy útil para el desarrollo del proyecto. Además, hay gran cantidad de recursos gráficos gratuitos ofrecidos para su utilización en Tiled, lo cual se adapta perfectamente a lo que queremos hacer, que incluye la implementación en el proyecto de gráficos pero no su creación.

Capítulo 2

Planificación y presupuestos

2.1. Gestión de configuración

2.1.1. Infraestructura del proyecto

Los proyectos de Love2D no siguen una estructura específica para su desarrollo, siendo estos una carpeta que contiene todos los archivos necesarios para el programa en organización de libre elección para el desarrollador. Debido a que nuestro programa busca relacionar sus distintas capas para que se afecten unas a otras (EJ: la victoria en un combate afecta al dueño de una región en la gestión de regiones), hemos decidido una organización de archivos que los junta basados en su tipo y no al módulo al que pertenecen, de manera que los archivos de código de inicio del juego estarán en nuestra carpeta principal, y los archivos secundarios (música, gráficos, etc) y código que manipula el funcionamiento del juego se guardarán en carpetas independientes, quedando finalmente la siguiente estructura de carpetas:

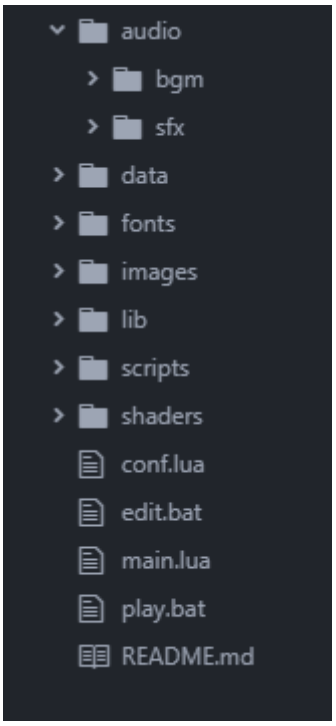

Figura 2.1: Estructura de carpetas

Los contenidos de cada carpeta incluyen:
Audio: Archivos de audio, separados en bandas sonoras (BGM) y efectos de sonido (SFX).
Data: Archivos de datos, donde guardamos código que define estructuras constantes del juego, como los archivos de matrices de los mapas, o estadísticas de unidades.
Fonts: Archivos para el uso de fuentes en texto.
Images: Imágenes del programa, incluyendo iconos, sprites, VFX, terreno...
Lib: Librerías y DLL necesarios para el funcionamiento de Love2D
Scripts: Archivos de código de los que depende la funcionalidad del juego, tenemos aquí todas las funciones que rigen el juego, pero también los diálogos que se asignan a NPCs, asignaciones de SFX y VFX a partes de la interfaz, etc.
Shaders: Los shaders son donde guardamos los archivos de modificación de gráficos, que están dedicados la transformación de sprites para sus distintas opciones (brillo, saturación, etc), pero también a la modificación de ciertos elementos GUI, como el uso de bordes para texto.

Conf.lua y main.lua son los archivos de configuración e inicio del programa. Los archivos .bat son scripts para el inicio del programa(play) y editor de texto(edit) y no serán incluidos en la entrega final.

2.1.2. Control de cambios

Debido a ser un proyecto personal e individual, el control de cambios fue relativamente laxo, pero siguió una estructura determinada a lo largo del desarrollo. El objetivo de esta estructura era, debido a la naturaleza más libre del proyecto, poder separar las distintas versiones de manera que todas ellas tengan cambios de funcionalidad bien marcados, de manera que si por ejemplo, de una versión a otra añadimos aspectos de personalización, y de esta nueva a la siguiente añadimos gráficos de mapa, podemos fácilmente replicar los cambios realizados a gráficos de mapa incluso si más adelante queremos volver a la versión en la que cambiamos la personalización para rehacer aspectos de esta.

Para el control de cambios, decidimos asignar ciertos puntos a cada fragmento del programa, y generar una nueva versión del mismo cada vez que se acumulaban 20 puntos o se completaba un módulo (ya que estos son las partes principales del programa), y asignamos los puntos a cada fragmento dependiendo de su importancia al programa y no su complejidad, ya que la razón más frecuente por la que se podría querer volver a versiones anteriores es el cambio de una función principal del juego por haber encontrado mejores maneras de implementarla. Se definen los siguientes fragmentos posibles en los que dividir el programa:

Pieza de contenido: Las piezas de contenido son piezas pequeñas del programa que forman parte de una funcionalidad o módulo. Estas incluyen instancias individuales de menús, funciones que forman parte de una habilidad, etc. A estas piezas asignamos 1 punto.

Funcionalidad: Llamamos funcionalidad a un aspecto principal del funcionamiento del juego, como menús, ejecución de combates, cambios de clase, etc. A las funcionalidades asignamos 5 puntos

Submódulos: Llamamos submódulos a las funcionalidades de interacción entre módulos completos, como son la toma de información del módulo de gestión de unidades para generar una instancia de combate. Los submódulos son generalmente simples, pero debido a su importancia para el funcionamiento del juego, les asignamos 10 puntos.

Módulo: Los 3 módulos principales del juego son el combate, la gestión de regiones y la gestión de unidades. Siempre generaremos una nueva versión al completar uno de ellos.

2.2. Gestión de riesgos

Definimos los riesgos del proyecto como condiciones inciertas, basadas ya sea en falta de experiencia o conocimientos del desarrollador, o en condiciones fuera del control del mismo. Aunque técnicamente existen los riesgos positivos y pueden ser gestionados, nos centraremos en los riesgos negativos, que son la amenaza principal al desarrollo de nuestro proyecto.

2.2.1. Clasificación de los factores de riesgo

Para la clasificación de riesgos, utilizaremos 3 factores principalmente:

Probabilidad:

Baja: Consideramos un riesgo como probabilidad baja, cuando es más probable que este no ocurra a lo largo del proyecto que que sí lo haga.

Alta: Un riesgo de probabilidad alta probablemente aparezca a lo largo de un proyecto, pero no es asegurado.

Segura: Un riesgo de probabilidad segura aparecerá en el proyecto con seguridad casi absoluta, y suelen englobar las partes más inciertas del proyecto.

Severidad:

Baja: Una severidad baja implica correcciones de pocas horas o incluso minutos. Engloban problemas de gestión menores o errores de código individuales

Media: Una severidad media implica un retraso de más de un día al proyecto, y puede englobar errores de código que afecten a varias secciones, o problemas de gestión mayores como pérdida de trabajo por no guardar apropiadamente archivos al final de una sesión de trabajo

Alta: Una severidad alta puede implicar un retraso de varios días o semanas, e incluye pérdidas de versiones completas del programa, o rediseños completos de parte del programa o documento.

Fatal: Una severidad fatal implica una pérdida masiva de trabajo debido a la pérdida del proyecto completo, errores de concepto que signifiquen tener que rehacer todo o gran parte del proyecto, o perdidas de secciones enteras del documento. También incluiríamos aquí riesgos que implicasen la imposibilidad de

entregar el proyecto en el plazo establecido que afecten al desarrollador (accidente, enfermedad, etc).

Impacto:

El impacto es el factor que determina la importancia de evitar o minimizar estos riesgos en el proyecto, y se obtiene como un producto de la severidad y probabilidad de un riesgo siguiendo la siguiente matriz:

Severidad / Probabilidad	Baja	Alta	Segura
Baja	Bajo	Bajo	Medio
Media	Bajo	Medio	Medio
Alta	Medio	Alto	Urgente
Fatal	Alto	Urgente	Fatal

Estos niveles de impacto tienen las siguientes implicaciones en el proyecto:

Bajo: Estos riesgos son menores, y generalmente ignorables, preocupándonos de ellos solo si son de solución fácil y rápida, o hemos lidiado con el resto de riesgos.

Medio: Estos riesgos son notables, y deberíamos tener planes de mitigación para ellos siempre que fuese posible.

Alto: Estos riesgos pueden afectar gravemente al desarrollo del proyecto y debemos tener un plan de acción para todos ellos, ya sea para mitigar o evitarlos.

Urgente: Estos riesgos deberían ser gestionados con mucho cuidado tanto para mitigarlos como para evitarlos en medida de lo posible, y deberían causar un replanteamiento del proyecto si aparecen en gran número.

Fatal: Un riesgo fatal es causa suficiente para replantear el proyecto de manera que este no aparezca, a no ser que se tengan planes extremadamente sólidos para su gestión, o sean inevitables por circunstancias externas a nuestro control.

Generalmente, podemos aceptar los riesgos de bajo impacto, y tal vez algunos de medio impacto, sobre todo cuando el beneficio de evitarlos no supera al coste de la solución (generalmente en riesgos de probabilidades altas pero severidad baja, que serán difíciles de evitar a pesar de no afectarnos demasiado), pero cualquier riesgo de impacto alto o mayor debe tener planes de gestión.

2.3. Clasificación de los riesgos

2.3.1. Planes de respuesta

Los planes de respuesta a los riesgos se separan en 3 categorías:

Planes de prevención: Estos son los planes que se toman para poder evitar que ocurran los riesgos, y afectan principalmente a su probabilidad de ocurrir. Esto incluye cosas como mantenimiento de un equipo informático para evitar el riesgo de que se estropee.

Planes de mitigación: Estos son los planes que se preparan como respuesta a un riesgo cuando este ocurre para mitigar su impacto. Esto incluiría tener un equipo de reserva para mitigar el riesgo de que se estropee el equipo informático que estamos utilizando.

Planes de transferencia: Estos planes consisten en la transferencia de la responsabilidad del riesgo a un tercero para aligerar la carga del responsable. Debido a ser un proyecto individual, este plan solo se podrá tomar en condiciones excepcionales, donde se puedan transferir al tutor ciertas responsabilidades o obtener apoyo de familia o amigos para riesgos personales.

2.3.2. Datos de clasificación de riesgos

En esta sección se incluyen los datos y categorías que incluirán las tablas de análisis de riesgos. La lista completa de datos a especificar es la siguiente:

DATOS

ID: Asignaremos un ID con formato RSC-XX a todos los riesgos basados en el orden en que fueron documentados.

Nombre: Asignaremos a cada riesgo un nombre, que será una frase corta que describa lo que implica a grandes rasgos.

Descripción: Cada riesgo incluirá una descripción más detallada donde especificamos el problema que implica.

Categoría: Cada riesgo será encuadrado en una de las siguientes categorías:

Personal: Este riesgo afecta o depende del desarrollador del proyecto. Aquí se incluye enfermedad, problemas familiares/personales, etc.

Planificación: Aquí entran los riesgos relacionados con mala planificación, o situaciones externas que afectan negativamente a la planificación realizada.

Desarrollo: Aquí entran riesgos que afecten al desarrollo del propio programa, como problemas con el motor o lenguaje utilizados.

Documentación: Aquí entran los riesgos relacionados con la documentación, que serán mayormente los relacionados con problemas que obliguen a rehacer secciones de los documentos, como cambios en el programa que impliquen cambiar manuales de usuario.

Impacto, probabilidad y severidad: Se clasificarán los riesgos como se mencionó anteriormente.

PLANES DE RESPUESTA

Tipo: Separamos las acciones en preventivas o de mitigación. La de transferencia se incluirán en mitigación.

Acción: Se detalla la acción tomada como parte del plan de respuesta.

Coste: Aquí incluimos los costes económicos y/o temporales de la acción tomada/a tomar.

Indicar: Aquí se detalla a que indicadores debemos prestar atención para determinar que hay que poner en marcha el plan de respuesta

ESTIMACIÓN DE RESULTADO

Aquí incluiremos los nuevos valores de impacto, probabilidad y severidad estimados teniendo en cuenta el plan de respuesta. Aquí se permitirá la inclusión del valor "Nulo"para las 3 categorías, para casos en los que el plan de respuesta garantice la anulación del riesgo.

RESULTADOS

Esta sección se completará cuando el proyecto esté finalizado, y detalla los resultados finales de la gestión de riesgos mediante las siguiente categorías:

Manifestación: Aquí indicamos si el riesgo acabó manifestándose a lo largo del proyecto, si este campo es negativo, el resto de campos de Resultados"no aplicarán.

Consecuencias: Aquí se indican las consecuencias que se dieron debido a la manifestación del riesgo.

Efectos de respuesta: Aquí indicamos el efecto que resultó de aplicar nuestro plan de respuesta al riesgo. Se aplica sobre todo a riesgos con plan de mitigación, no de prevención.

Comentarios: Comentarios adicionales del desarrollador sobre este riesgo.

2.3.3. Tablas de riesgos iniciales

En esta sección de incluyen las tablas de riesgos de los riesgos identificados desde el principio del proyecto.

RSC-01		**Aprendizaje más largo de lo esperado**
Descripción		La fase de aprendizaje dura más tiempo del estimado en la planificación temporal.
Categoría		Planificación
Análisis actual del riesgo		
Probabilidad		Alta
Severidad		Media
Impacto		Medio
Plan de respuesta		
Prevención	**Acción**	Ninguna
	Coste	No aplica
Mitigación	**Acción**	Se deja un margen en la planificación para los posibles retrasos.
	Coste	La planificación es más difícil por querer dejar tiempo en reserva
Indicador		Si nos pasamos de la fecha, o nos acercamos mucho, y aún está sin completar el aprendizaje sin señal de que vaya a completarse pronto.
Estimación del resultado		
Probabilidad		Alta
Severidad		Baja
Impacto		Bajo
Resultados		
Manifestación y consecuencias		El riesgo se manifestó, lo cual consumió parte del tiempo adicional preparado para posibles retrasos
Efectos de respuesta		La respuesta finalmente se mostró innecesaria, pues se manifestó un riesgo positivo (finalizamos mucho más rápido el desarrollo).
Comentarios		Se puede considerar que el riesgo no se manifestó, sinó que se hizo una modificación inesperada de la planificación, ya que el mayor tiempo de aprendizaje se dedicó a herramientas que más adelante aceleraron mucho el tiempo de desarrollo.

RSC-02		**Planificación optimista**
Descripción		La planificación realizada es demasiado optimista al asignar la duración de cada tarea.
Categoría		Planificación
Análisis actual del riesgo		
Probabilidad		Baja
Severidad		Media
Impacto		Bajo
Plan de respuesta		
Prevención	**Acción**	Ninguna
	Coste	No aplica
Mitigación	**Acción**	Se deja un margen en la planificación para los posibles retrasos
	Coste	La planificación es más difícil por querer dejar tiempo en reserva
Indicador		Alguno de los plazos de la planificación no ha sido cumplido o nos estamos acercando a fin de plazo sin un progreso adecuado
Estimación del resultado		
Probabilidad		Baja
Severidad		Baja
Impacto		Bajo
Resultados		
Manifestación y consecuencias		El riesgo se manifestó en ciertas partes de la planificación, sin embargo, la planificación fue pesimista en otras partes, por lo que acabó equilibrándose.
Efectos de respuesta		No fue necesario utilizar el tiempo reservado.
Comentarios		Ninguno.

RSC-03		**Inicio lento**
Descripción		Las actividades planificación nos llevan más tiempo del esperado o se inician tarde.
Categoría		Documentación
Análisis actual del riesgo		
Probabilidad		Baja
Severidad		Alta
Impacto		Medio
Plan de respuesta		
Prevención	**Acción**	Se marcan fechas precisas como límite para el inicio de las actividades.
	Coste	Externo al proyecto
Mitigación	**Acción**	Ninguna
	Coste	No aplica
Indicador		Las tareas de planificación no están echas con los 4 meses de antelación que suelen asignarse a un TFG.
Estimación del resultado		
Probabilidad		Nula
Severidad		Alta
Impacto		Nulo
Resultados		
Manifestación y consecuencias		Este riesgo no se manifestó
Efectos de respuesta		Se aseguró que el riesgo no se manifestase
Comentarios		Ninguno.

RSC-04		**Diseño inadecuado**
Descripción		El diseño no se adapta bien a las capacidades del lenguaje y motor escogidos
Categoría		Desarrollo
Análisis actual del riesgo		
Probabilidad		Alta
Severidad		Fatal
Impacto		Urgente
Plan de respuesta		
Prevención	**Acción**	Investigación temprana de las capacidades del lenguaje
	Coste	4 Horas
Mitigación	**Acción**	Investigación de posibles tecnologías alternativas
	Coste	3 Horas Esperadas, 0 Horas necesitadas
Indicador		El desarrollador se ve incapaz de cumplir los requisitos con las tecnologías asignadas
Estimación del resultado		
Probabilidad		Nula
Severidad		Fatal
Impacto		Nulo
Resultados		
Manifestación y consecuencias		La investigación preventiva confirmó que se podrían cumplir los requisitos, y el desarrollador observó proyectos similares con las mismas tecnologías que lo confirmaban.
Efectos de respuesta		El plan de prevención garantizó que no ocurriese el riesgo, lo cual también nos evitó la necesidad de preparar el plan de mitigación
Comentarios		Ninguno.

RSC-05		**Código de baja calidad**
Descripción		El código de la aplicación ofrece un rendimiento inferior al esperado o da lugar a múltiples errores
Categoría		Desarrollo
Análisis actual del riesgo		
Probabilidad		Segura
Severidad		Variable (Media-fatal)
Impacto		Urgente
Plan de respuesta		
Prevención	**Acción**	Se planifica una fase de aprendizaje extensa para evitar riesgos de este tipo
	Coste	1 Semana añadida a la planificación inicial de aprendizaje
Mitigación	**Acción**	Preparación de material de aprendizaje adicional y ayuda de persona con conocimiento experto del lenguaje
	Coste	2 Horas
Indicador		La aplicación funciona excesivamente lenta o gran cantidad de los intentos de compilación muestran errores.
Estimación del resultado		
Probabilidad		Baja
Severidad		Alta
Impacto		Medio
Resultados		
Manifestación y consecuencias		El riesgo no se manifestó, en parte por la cantidad adicional de tiempo de aprendizaje y en parte por la gran cantidad y calidad de material educativo del lenguaje
Efectos de respuesta		El plan ayudó de manera efectiva a prevenir el riesgo.
Comentarios		Ninguno.

RSC-06		**Indisposición**
Descripción		El desarrollador se encuentra indispuesto por enfermedad o accidente.
Categoría		Personal
Análisis actual del riesgo		
Probabilidad		Baja
Severidad		Fatal
Impacto		Alto
Plan de respuesta		
Prevención	**Acción**	Ninguna
	Coste	No aplica
Mitigación	**Acción**	Se intentará cumplir los plazos del trabajo con antelación para que posibles imprevistos no impidan completar el proyecto
	Coste	No aplica
Indicador		Riesgo evidente.
Estimación del resultado		
Probabilidad		Baja
Severidad		Alta
Impacto		Media
Resultados		
Manifestación y consecuencias		El riesgo no se manifestó
Efectos de respuesta		No aplica
Comentarios		Este es un riesgo difícil de tener en cuenta por lo impredecible que es, por lo que sus características son aproximadas.

RSC-07		**Requisitos mal definidos**
Descripción		Los requisitos del programa son mal definidos por el desarrollador.
Categoría		Planificación/desarrollo.
Análisis actual del riesgo		
Probabilidad		Segura
Severidad		Alta
Impacto		Urgente
Plan de respuesta		
Prevención	**Acción**	Ninguno
	Coste	No aplica
Mitigación	**Acción**	Se realiza la planificación con requisitos cambiantes en mente
	Coste	Solo afecta a la planificación
Indicador		Se cumplen los requisitos pero no cumplen las expectativas iniciales, los requisitos aparentan imposibles de completar o se alejan de la visión global del proyecto.
Estimación del resultado		
Probabilidad		Segura
Severidad		Nulo
Impacto		Nulo
Resultados		
Manifestación y consecuencias		El riesgo se manifestó pero fue falto de consecuencias
Efectos de respuesta		Debido a la naturaleza del proyecto y la planificación realizada los requisitos cambiantes no solo eran de esperar sino intencionales, lo cual los elimina como posible riesgo
Comentarios		Debido a que el desarrollador busca llevar al proyecto de acuerdo a una visión personal, la planificación está realizada con requisitos cambiantes en mente para adaptarse a la visión del desarrollador, lo que hace que este riesgo sea mayormente irrelevante.

2.3.4. Tablas de riesgos progresivas

En esta sección estarán las tablas de riesgos de los riesgos que fueron identificados a lo largo del proyecto, una vez ya completa la tabla de riesgos iniciales.

RSC-08		**Falta de tiempo**
Descripción		Debido a la toma de otros proyectos u obligaciones, no se pueden cumplir las demandas temporales de la planificación.
Categoría		Personal/planificación
Análisis actual del riesgo		
Probabilidad		Baja
Severidad		Alta
Impacto		Medio
Plan de respuesta		
Prevención	**Acción**	Evitar la toma de proyectos adicionales durante el tiempo de trabajo en este proyecto
	Coste	No aplica
Mitigación	**Acción**	Limitar la cantidad de tiempo invertida en los nuevos proyectos, o compensar con tiempo adicional en el futuro
	Coste	No aplica
Indicador		Debido a otras obligaciones, el desarrollador se ve incapaz de dedicar el tiempo planificado al proyecto durante varios días seguidos
Estimación del resultado		
Probabilidad		Baja
Severidad		Baja
Impacto		Bajo
Resultados		
Manifestación y consecuencias		El riesgo se manifestó, no a pesar de los planes de respuesta, sino por no haberlos ejecutado adecuadamente. El desarrollador tomó nuevas responsabilidades para las que no tenía el tiempo adecuado y fue ralentizado el desarrollo de este proyecto.
Efectos de respuesta		Tras la finalización de los proyectos adicionales, se le dedicó más tiempo del planificado a este para compensar, lo que permitió devolverlo a su curso.
Comentarios		La responsabilidad adicional tomada fue un empleo, lo cual impidió dedicar el tiempo adecuado a este proyecto y a las responsabilidades escolares previas.

RSC-09		**Afectado por COVID-19**
Descripción		El desarrollador se ve afectado por la epidemia de COVID-19, ya sea por enfermedad o impedimento de ciertas acciones.
Categoría		Personal
Análisis actual del riesgo		
Probabilidad		Alta
Severidad		Fatal
Impacto		Urgente
Plan de respuesta		
Prevención	**Acción**	Se toman las medidas indicadas por el gobierno de distanciación social y evitan las salidas innecesarias.
	Coste	Personal, no afecta al proyecto.
Mitigación	**Acción**	Cuando las salidas son necesarias, se utiliza máscara. Si se ven afectadas reuniones o tutorías, se buscan maneras alternativas de realizarlas.
	Coste	Coste de mascarillas desestimado por ser necesarias incluso fuera del proyecto.
Indicador		El desarrollador es contagiado o ve sus actividades impedidas por las restricciones en lugar.
Estimación del resultado		
Probabilidad		Baja
Severidad		Alta
Impacto		Medio
Resultados		
Manifestación y consecuencias		El riesgo se manifestó en su forma menos impactante, al impedir reuniones y tutorías en personas.
Efectos de respuesta		Se prepararon maneras alternativas de comunicación, lo que permitió continuar las reuniones con solo pequeños retrasos.
Comentarios		Parte de la respuesta fue permitida por organismos de gobierno o universidad, como la disponibilidad de las cuentas universitarias para Microsoft Teams.

2.4. Gestión temporal

Debido a la estructura modular del proyecto, y que estos módulos tienen funcionalidad parcial independiente pero dependen unos de otros para su funcionalidad completa, el proyecto será planificado mediante un ciclo de vida por

prototipos.

2.4.1. Cronograma

		Nombre	Duracion	Inicio	Terminado	Predecesores
1		Inicio del proyecto	0 days	13/03/19 8:00	13/03/19 8:00	
2		**Fase de aprendizaje y preparación**	21 days	**13/03/19 8:00**	**10/04/19 17:00**	**1**
3		Aprendizaje de programación IA	10,5 days	13/03/19 8:00	27/03/19 13:00	
4		Aprendizaje de programación gráfica	7,5 days	27/03/19 13:00	5/04/19 17:00	3
5		Aprendizaje de elaboración de sprites	3 days	8/04/19 8:00	10/04/19 17:00	4
6		**Diseño y planificación del prototipo**	10 days	**11/04/19 8:00**	**24/04/19 17:00**	**2**
7		Diseño de jugabilidad	3 days	11/04/19 8:00	15/04/19 17:00	
8		Diseño técnico del proyecto	5 days	16/04/19 8:00	22/04/19 17:00	7
9		Creación formal de plan de proyecto	2 days	23/04/19 8:00	24/04/19 17:00	8
10		**Desarrollo del prototipo**	25 days	**25/04/19 8:00**	**29/05/19 17:00**	**6**
11		Desarrollo sistema de unidades	3 days	25/04/19 8:00	29/04/19 17:00	
12		Desarrollo del mapa estratégico	6 days	30/04/19 8:00	7/05/19 17:00	11
13		Desarrollo del mapa táctico	6 days	8/05/19 8:00	15/05/19 17:00	12
14		Implementación de módulos	10 days	16/05/19 8:00	29/05/19 17:00	12;11;13
15		**Análisis del proceso de desarrollo y elaboración del informe**	14 days	**30/05/19 8:00**	**18/06/19 17:00**	**10**
16		Análisis del proceso de desarrollo e informe básico	7 days	30/05/19 8:00	7/06/19 17:00	
17		Elaboración de informe formal de TFG a partir del informe base	7 days	10/06/19 8:00	18/06/19 17:00	16
18		**Revisión y correcciónes finales del proyecto**	7 days	**19/06/19 8:00**	**27/06/19 17:00**	**15**
19		Revisión del prototipo	4 days	19/06/19 8:00	24/06/19 17:00	
20		Revisión del informe de TFG	3 days	25/06/19 8:00	27/06/19 17:00	19
21		**Elaboración de presentación y preparación para defensa del TFG**	10 days	**28/06/19 8:00**	**11/07/19 17:00**	**18**
22		Elaboración de la presentación	5 days	28/06/19 8:00	4/07/19 17:00	
23		Practicas exposición de la presentación	3 days	5/07/19 8:00	9/07/19 17:00	22
24		Análisis del proyecto en preparación para posibles preguntas	2 days	10/07/19 8:00	11/07/19 17:00	23

Figura 2.2: Cronograma inicial

2.4.2. Hitos del proyecto

Debido a nuestro ciclo de vida, hemos decidido que cada prototipo funcional sea uno de los hitos definidos, obteniendo la siguiente planificación:

ID	Nombre	Prototipo
H-01	Tareas preliminares	Ninguno, este paso se basa en documentación
H-02	Gestión de unidades	Gestor con unidades programadas y las funcionalidades básicas
H-03	Gestión de regiones	Inclusión del mapamundi y funciones de construcción y economía
H-04	Módulo de combate	Combates fijos no personalizables con unidades predeterminadas
H-05	Interacción combate-unidades	Modificación del combate para poder personalizar las unidades antes de su comienzo
H-06	Interacción unidades-región	Adición al mapamundi de las funciones de gestión de ejércitos.
H-07	Interacción combate-región	Adición al mapamundi de las funciones de invasión y conquista de regiones
H-08	Proyecto final	Proyecto e interacción entre prototipos completa.

2.4.3. Evolución del cronograma

Tras la realización del proyecto, pudimos observar que el tiempo que hubo que dedicar a aprendizaje y documentación fue bastante mayor del inicialmente estimado, mientras que el desarrollo fue más rápido gracias a las herramientas de asistencia que trabajan en conjunto con Love2D, cuya introducción al proyecto fue lo que aumentó el tiempo de aprendizaje. Esto resultó en alteraciones al cronograma, y la versión final puede encontrarse aqui:

		Nombre	Duracion	Inicio	Terminado	Predecesores
1		Inicio del proyecto	0 days	13/03/19 8:00	13/03/19 8:00	
2		**Fase de aprendizaje y preparación**	**28 days**	**13/03/19 8:00**	**19/04/19 17:00**	**1**
3		Aprendizaje de programación IA	10,5 days	13/03/19 8:00	27/03/19 13:00	
4		Aprendizaje de programación gráfica	7,5 days	27/03/19 13:00	5/04/19 17:00	3
5		Aprendizaje de elaboración de sprites	3 days	8/04/19 8:00	10/04/19 17:00	4
6		Aprendizaje de lenguaje Lua	7 days	11/04/19 8:00	19/04/19 17:00	5
7		**Diseño y planificación del prototipo**	**10 days**	**22/04/19 8:00**	**3/05/19 17:00**	**2**
8		Diseño de jugabilidad	3 days	22/04/19 8:00	24/04/19 17:00	
9		Diseño técnico del proyecto	5 days	25/04/19 8:00	1/05/19 17:00	8
10		Creación formal de plan de proyecto	2 days	2/05/19 8:00	3/05/19 17:00	9
11		**Desarrollo del prototipo**	**16 days**	**6/05/19 8:00**	**27/05/19 17:00**	**7**
12		Desarrollo sistema de unidades	3 days	6/05/19 8:00	8/05/19 17:00	
13		Desarrollo del mapa estratégico	5 days	9/05/19 8:00	15/05/19 17:00	12
14		Desarrollo del mapa táctico	2 days	16/05/19 8:00	17/05/19 17:00	13
15		Implementación de módulos	8 days	20/05/19 8:00	27/05/19 17:00	13;12;14
16		**Análisis del proceso de desarrollo y elaboración del informe**	**14 days**	**28/05/19 8:00**	**14/06/19 17:00**	**11**
17		Análisis del proceso de desarrollo e informe básico	7 days	28/05/19 8:00	5/06/19 17:00	
18		Elaboración de informe formal de TFG a partir del informe base	7 days	6/06/19 8:00	14/06/19 17:00	17
19		**Revisión y correcciónes finales del proyecto**	**7 days**	**17/06/19 8:00**	**25/06/19 17:00**	**16**
20		Revisión del prototipo	4 days	17/06/19 8:00	20/06/19 17:00	
21		Revisión del informe de TFG	3 days	21/06/19 8:00	25/06/19 17:00	20
22		**Elaboración de presentación y preparación para defensa del TFG**	**10 days**	**26/06/19 8:00**	**9/07/19 17:00**	**19**
23		Elaboración de la presentación	5 days	26/06/19 8:00	2/07/19 17:00	
24		Practicas exposición de la presentación	3 days	3/07/19 8:00	5/07/19 17:00	23
25		Análisis del proyecto en preparación para posibles preguntas	2 days	8/07/19 8:00	9/07/19 17:00	24

Figura 2.3: Nuevo Cronograma

2.5. Gestión de costes

Debido a la naturaleza individual del proyecto, los costes son bastante mínimos, pero se estiman los siguientes costes materiales y de recursos humanos:

2.5.1. Costes de recursos humanos

Para el cálculo de costes de la USC en sueldo de tutor utilizamos las tablas salariales del servicio de gestión de personal de la USC [1]. Para los costes totales del contrato, se utilizó la calculadora de contratos proporcionada por la USC. El desarrollador no tiene como tal sueldo, pero si incurre costes para trabajar en el proyecto, que se tendrán en cuenta en la sección de costes indirectos.

El coste de sueldo final calculado es de 25€por hora. Siguiendo las horas de tutorías asignadas al TFG como horas presenciales (11.25 horas de trabajo presencial, que incluyen la evaluación, nos dejan aproximadamente 10.5 horas en tutorías). Esto nos dá un coste de 25*10.5=262.5€.

2.5.2. Costes materiales

Los costes materiales de este proyecto incluyen costes de licencias de software, pero estos fueron mínimos debido a que la mayoría de software utilizado es gratuito y de uso libre. Podemos ver en la siguiente tabla los costes materiales finales, calculados basándonos en su precio, sus horas de uso y sus horas de vida útil:

Recurso	Horas de uso	Coste/H	Coste total
Portatil personal	350	0.18€	63€
Licencia de Windows	350	0.01€	3.5€
Acceso a internet	240	0.1€	24€

2.5.3. Costes indirectos

Los costes indirectos son los que no podemos trazar directamente a los recursos o activos del proyecto, sino que dependen de su entorno, como agua, electricidad, mobiliario, etc. Estos costes suelen constar como un aumento porcentual de los costes totales, y los propios datos de la USC nos recomiendan un 21 % para proyectos de ingeniería informática.

2.5.4. Costes totales

El cálculo de costes finales es muy sencillo, cogemos el total de costes materiales y de recursos humanos: 262.5+63+3.5+24=353€. A esto aplicamos el porcentaje asignado de costes indirectos, y obtenemos el coste total del proyecto: 353*1.21= 427.13€de coste total.

2.5.5. Coste alternativo

A pesar de que no el desarrollador del proyecto no recibe un sueldo por el desarrollo del TFG, podríamos considerar que sus horas de trabajo se están haciendo a una tasa de ingeniero novato, pues el TFG es lo que le çonfirmaçomo tal, y ha recibido ya la formación en el grado. En tal caso, usando de nuevo la tabla de sueldos de la USC, una búsqueda rápida en Glassdoor [2] nos muestra sueldos promedios de 27000 euros anuales aproximados. Debido a que nuestro nivel de experiencia equivalente es recién graduados, cogeremos el rango bajo de estos sueldos, que nos ponen en aproximadamente 20000€al año. Aproximando meses de 170 horas (40 horas semanales y meses de 30 días, aproximamos a 170 porque nos facilita la división de las 410 horas de trabajo de un TFG), tenemos aproximadamente 3 meses de paga, lo cual nos aproxima un sueldo bruto de 20000/4 = 5000€aproximados.
Si nos decidimos por cálculo de costes alternativo, los costes totales del proyecto ascenderían a 427.13 + 5000*1.21 = 6477.13€.

2.6. Gestión de comunicaciones

La gestión de comunicaciones, debido a la naturaleza del proyecto, es mayormente irrelevante. Simplemente detallamos los distintos niveles de comunicación que hubo entre el alumno y tutor para distintas situaciones:

-Para dudas rápidas, se enviaron correos mediante el servicio de correo universitario.

-Para revisiones de TFG, inicialmente se hicieron en persona, tras previa cita por correo electrónico, pero debido a la situación de pandemia por Covid-19, las reuniones cercanas al final del proyecto se organizaron a través de Microsoft Teams, también utilizando las cuentas universitarias, como se indica en la tabla de riesgos progresiva RSC-09.

Capítulo 3

Especificación de requisitos

3.1. Requisitos del proyecto

3.1.1. Requisitos funcionales

En este apartado se detallarán los requerimientos funcionales del proyecto. Estos han sido escogidos por el desarrollador a partir de sus propias ideas para el resultado final del proyecto, y la consulta de amigos del desarrollador sobre que les gustaría ver en juegos del genero. Estos requisitos se organizarán en tablas, cuyos campos contienen características de estos requisitos.

ID: Código que identifica el requisito de forma RQF-XX. Se asigna por orden de creación y debe ser único.
Nombre: Nombre del requisito, debe ser lo más descriptivo posible sin extenderse en exceso.
Descripción: Descripción exacta del requisito, que debe dejar claro lo necesario para que este se considere cumplido.
Importancia: Valora la importancia del cumplimiento del requisito mediante los valores: vital, importante y quedaría bien [IBM97].
Estabilidad: Nos indica la posibilidad de que este requisito sea cambiado a medida que se trabaja en el proyecto. Utiliza los valores: fija(El requisito es una parte central del proyecto y no recibirá ningún cambio), alta(el requisito es una parte importante del proyecto que puede pasar por cambios menores, pero cuya idea central será fija), media(la ideal principal del requisito se mantendrá en gran parte, pero puede sufrir cambios o matices importantes) o baja(el requisito ofrece una idea general que puede cambiar mucho antes de llegar a su forma final).
Requisitos previos: Aquí se incluyen, si es relevante, los requisitos que deben ser completados previos a este por depender del anterior.

RQF-01	**Combate táctico**
Descripción	La aplicación debe ofrecer la posibilidad de iniciar combates tácticos en mapas de cuadrículas, donde dos ejércitos formados por unidades se enfrentan por el control de una región.
Importancia	Vital
Estabilidad	Fija
Requisitos previos	No

RQF-02	**Gestión de unidades**
Descripción	La aplicación debe ofrecer al jugador un menú de gestión donde pueda ver a las unidades bajo su control y personalizarlas de distintas maneras.
Importancia	Vital
Estabilidad	Fija
Requisitos previos	No

RQF-03	**Gestión de equipamiento**
Descripción	La aplicación debe permitirnos cambiar el equipamiento de las unidades bajo nuestro control en el menú de gestión por cualquier otro equipamiento en nuestra posesión que pertenezca a su ranura adecuada.
Importancia	Vital
Estabilidad	Fija
Requisitos previos	RQF-02

RQF-04	**Cambio de clase**
Descripción	La aplicación debe permitirnos cambiar la clase a la que pertenecen las unidades bajo nuestro control en el menú de gestión siempre que estas cumplas los requisitos de la clase a la que desean cambiar.
Importancia	Vital
Estabilidad	Fija
Requisitos previos	RQF-02

RQF-05	**Personalización de habilidades**
Descripción	La aplicación debe permitirnos cambiar las habilidades equipadas de las unidades bajo nuestro control en el menú de gestión de manera que puedan tener equipadas cualquier habilidad que hayan aprendido siempre que no superen su límite de cantidad.
Importancia	Vital
Estabilidad	Alta
Requisitos previos	RQF-02

RQF-06	**Observación de unidades**
Descripción	La aplicación debe permitirnos ver, desde el menú de gestión de unidades, las características de cada unidad que hay bajo nuestro control.
Importancia	Vital
Estabilidad	Fija
Requisitos previos	RQF-02

RQF-07	**Orden de unidades**
Descripción	La aplicación debe permitir, en el menú de gestión de unidades, cambiar el orden en el que aparecen las distintas unidades bajo nuestro control.
Importancia	Quedaría bien
Estabilidad	Baja
Requisitos previos	RQF-02

RQF-08	**Títulos**
Descripción	La aplicación debe ofrecer al jugador una lista de requisitos exclusiva a cada unidad de manera que cuando la unidad cumpla cada requisito obtenga un emblema que afecte a sus características o desbloquee acceso a nuevas clases.
Importancia	Importante
Estabilidad	Media
Requisitos previos	RQF-02

RQF-09	**Recompensas de combate**
Descripción	La aplicación debe ofrecer al jugador una recompensa de objetos y/u oro después de un combate si ha obtenido la victoria.
Importancia	Quedaría bien
Estabilidad	Alta
Requisitos previos	RQF-01

RQF-10	**Gestión de regiones**
Descripción	La aplicación debe permitir al jugador acceder a los datos de regiones y mover las unidades bajo su control en una región a regiones adyacentes a esta, aunque estén bajo el control de un enemigo.
Importancia	Vital
Estabilidad	Fija
Requisitos previos	No

RQF-11	**Invasión de regiones**
Descripción	La aplicación debe permitir al jugador iniciar una invasión al mover unidades bajo su control a una región bajo control del enemigo.
Importancia	Vital
Estabilidad	Alta
Requisitos previos	RQF-01,RQF-02

RQF-12	**Conquista de regiones**
Descripción	La aplicación debe otorgar al jugador control de una región si ha obtenido victoria en un combate iniciado al invadir dicha región.
Importancia	Vital
Estabilidad	Alta
Requisitos previos	RQF-01,RQF-02,RQF-11

RQF-13	Progresión de nivel
Descripción	Las unidades del jugador deben poder acumular experiencia al tomar acciones en combate, cuando la experiencia una unidad llegue a un total de 100, esta debe subir de nivel, aumentando sus características.
Importancia	Vital
Estabilidad	Fija
Requisitos previos	RQF-01

RQF-14	Despliegue de unidades
Descripción	El jugador debe poder desplegar a cada unidad en un combate en cualquiera de las casillas iniciales permitidas, hasta llegar al límite de unidades del mapa.
Importancia	Importante
Estabilidad	Media
Requisitos previos	RQF-01,RQF-02

RQF-15	Progreso de regiones
Descripción	La aplicación debe permitir al jugador gastar su oro y materiales en la construcción de estructuras que ofrecen bonificaciones a las regiones en las que se construyen.
Importancia	Quedaría bien
Estabilidad	Alta
Requisitos previos	RQF-10

RQF-16	Control de unidades
Descripción	El jugador debe poder controlar el movimiento y utilización de habilidades de sus unidades en combate.
Importancia	Vital
Estabilidad	Fija
Requisitos previos	RQF-01

RQF-17	Sistema de guardado
Descripción	El jugador debe poder guardar la partida, de manera que continúe donde lo dejó previamente la próxima vez que inicie el programa
Importancia	Quedaría bien
Estabilidad	Fija
Requisitos previos	No

RQF-18	Progreso temporal
Descripción	El sistema debe incluir un sistema de tiempo de manera que cada acción avance el calendario interno del juego.
Importancia	Quedaría bien
Estabilidad	Media
Requisitos previos	RQF-02

3.1.2. Requisitos no funcionales

Aquí clasificamos los requisitos no funcionales de nuestro proyecto, en tablas con el mismo formato que las de los requisitos funcionales. El ID de estos requisitos tendrá el formato RNF-01

RNF-01	Aplicación en Lua
Descripción	La aplicación debe estar desarrollada completamente en Lua adaptado al motor Love2D.
Importancia	Vital
Estabilidad	Fija
Requisitos previos	No

RNF-02	Rendimiento mínimo
Descripción	El rendimiento del juego debe ser suficiente para que no afecte negativamente a la jugabilidad del mismo.
Importancia	Importante
Estabilidad	Fija
Requisitos previos	No

RNF-03	Modularidad del software
Descripción	Los módulos en los que se basa la aplicación deben tener funcionalidad independiente, de manera que si se incluye un módulo nuevo solo se necesita comunicarlo con los ya existentes para su funcionamiento conjunto
Importancia	Importante
Estabilidad	Alta
Requisitos previos	No

RNF-04	Tiempo máximo de carga
Descripción	El tiempo máximo que debe tardar en iniciar el programa será de 10 segundos, mientras que las pantallas de transición deberían durar menos de 5 (idealmente menos de 2).
Importancia	Importante
Estabilidad	Fija
Requisitos previos	No

RNF-05	Distribución
Descripción	La aplicación debe ser distribuible en forma de archivo ejecutable, zip, o idealmente instalador.
Importancia	Vital
Estabilidad	Alta
Requisitos previos	No

3.1.3. Requisitos de información

En esta sección describimos los requisitos de información de cada una de las distintas entidades del juego. Cada tabla incluirá las siguientes categorías:

ID: Código que identifica el requisito de forma RI-XX. Se asigna por orden de creación y debe ser único.
Entidad: La entidad de la cual debemos guardar datos.
Datos: Lista de datos que necesitamos tener de la entidad especificada.
Requisitos: Indica para que requisitos funcionales son necesarios los datos de las entidades.

RI-01	**Unidades**
Datos	Nombre, clase, raza, nivel, experiencia, estadísticas, atributos, equipamiento, habilidades, títulos, ID interno, controlador.
Requisitos	RQF-01,RQF-02,RQF-03,RQF-04,RQF-05,RQF-06,RQF-08,RQF-13,RQF-16

RI-02	**Regiones**
Datos	Capacidad de unidades, estructuras, adyacencia, beneficios mensuales, conjunto de regiones original, mapa de combate asignado, dueño.
Requisitos	RQF-01,RQF-10,RQF-11,RQF-12,RQF-15

RI-03	**Mapas**
Datos	Casillas, dimensiones, región, efectos especiales actuales, estructuras existentes.
Requisitos	RQF-01,RQF-09,RQF-16

RI-04	**Archivos de guardado**
Datos	Todo RI-01 y RI-02, fecha en el juego, eventos en progreso, ID de archivo.
Requisitos	RQF-17

RI-05	Objetos
Datos	Nombre, propiedades, efectos especiales, requerimientos.
Requisitos	RQF-03

RI-06	Habilidades
Datos	Nombre, coste, descripción, efectos.
Requisitos	RQF-05

RI-07	Razas
Datos	Nombre, modificadores.
Requisitos	RQF-06

RI-08	Clases
Datos	Nombre, bases, habilidades, requerimientos.
Requisitos	RQF-05,RQF-06

3.1.4. Requisitos de hardware

Debido a que no tenemos herramientas de medición de rendimiento, es difícil establecer unos requisitos mínimos de hardware, sin embargo, podemos utilizar como base los requisitos mínimos y recomendados de Love2D, que servirán como base mínima para los videojuegos desarrollados con este. Podemos deducir de las librerías que utilizan y los sistemas para los que ofrecen instalables el siguiente mínimo:

-Tarjeta gráfica capaz de ejecutar OpenGL 2.1.[4] -Sistema operativo Windows Vista o mayor.

Debido a que los videojuegos de Love2D utilizan gráficos ligeros en 2D y el lenguaje es bastante ligero, los requerimientos de hardware no deberían ser muy elevados, pudiendo asegurar por experiencia propia que lo siguiente es suficiente:

-Sistema operativo Windows 7. -Tarjeta de vídeo Intel Core i5-3330. -Memoria RAM de 4GB.

3.2. Actores

En nuestro proyecto solo hay dos actores, ambos claramente definidos:

ACT-01	**Jugador**
Descripción	El jugador es el actor principal, el cual controla el juego y que determina las acciones de los personajes bajo su control.

ACT-02	**Sistema**
Descripción	El sistema es el controlador de todas las funciones internas del juego, y funciona a su vez de oponente del jugador, utilizando IA para controlar las unidades enemigas en contra del jugador.

3.3. Casos de uso

Los casos de uso de la aplicación son en general sencillos, y suele pertenecer a dos categorías: la navegación de menús, con múltiples variaciones de la misma acción, lo cual se tiende a tener una pequeña cantidad de casos de uso muy generales, lo cuales contienen una gran cantidad de excepciones para dichas variaciones; y el control del juego, el cual tiene casos de uso más largos e interconectados para establecer las cadenas de acciones que nos llevan de un caso de uso al siguiente.

Debido a que la mayoría de casos de uso implementan una parte o la totalidad de un requisito funcional, estos no incluirán los campos de importancia y prioridad, que se heredarán de los requisitos a los que pertenecen, debido a esto, es posible que algún caso de uso no esté programado en el proyecto final si no lo está el requisito que implementa. Además, tanto el jugador como en la máquina están implicados en la totalidad de los casos de uso, siendo el jugador el que los ejecuta, y el sistema el que hace que funcionen a bajo nivel, por lo que omitiremos el campo habitual de .ªctores".

Los datos finales de cada caso de uso que definiremos serán los siguientes:

ID: Código que identifica el caso de uso UC-XX. Se asigna por orden de creación y debe ser único.
Nombre: Nombre del caso de uso.
Precondiciones: Condiciones en las que debe estar el sistema antes del inicio

del caso de uso.
Postcondiciones: Condiciones en las que se queda el sistema una vez el caso de uso está completado.
Camino base: El camino habitual seguido al ejecutar el caso de uso.
Excepción-XX: Excepciones al camino base en el caso de uso, donde la XX indica el primer paso que no realizamos debido a pasar al primer paso de la excepción.
Comentarios: Comentarios del desarrollador sobre el caso de uso.

A partir de aquí incluiremos una imagen de las relaciones de los casos de uso, seguida de las tablas que los describen.

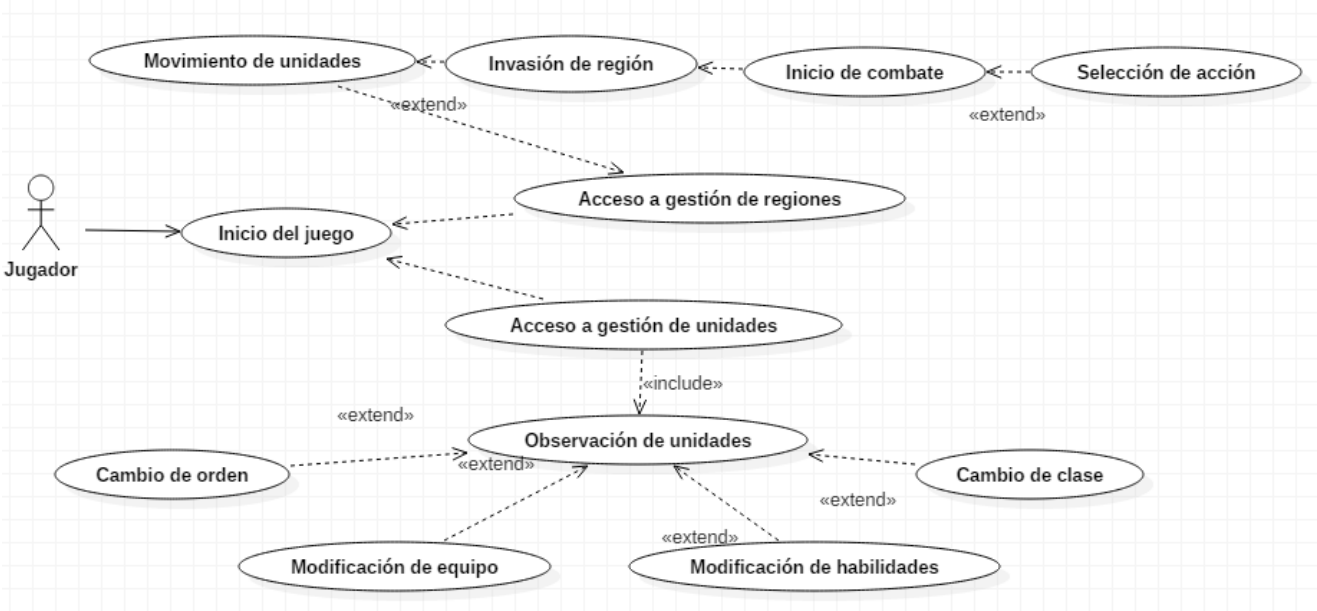

Figura 3.1: Casos de uso

UC-01	**Inicio del juego**
Precondiciones	Hemos obtenido el ejecutable del juego y los archivos necesarios para su funcionamiento.
Postcondiciones	El juego está iniciado en el menú principal.
Camino Base	
Paso 1	Iniciamos el ejecutable del juego
Paso 2	El sistema carga el menú inicial y nos muestra la pantalla de introducción.
Excepciones	
No aplica	No aplica.

UC-02	**Acceso a la gestión de regiones**
Precondiciones	Completado UC-01, nos encontramos en la pantalla del menú principal.
Postcondiciones	Nos encontramos en la pantalla de gestión de regiones.
Camino Base	
Paso 1	En el menú principal, mover el cursor a nueva partida y aceptar con Z, espacio, o enter.
Paso 2	El sistema realiza la carga del juego y nos lleva al mapamundi en estado de nueva partida.
Excepciones	
Paso 1	Si el jugador tiene una partida guardada, puede seleccionar cargar partida.
Paso 2	El sistema lee el archivo de guardado e inicia la partida en las condiciones que estaba cuando se efectuó el guardado.
Paso 3	El sistema nos lleva al mapamundi en las condiciones cargadas.

UC-03	**Movimiento de unidades**
Precondiciones	Nos encontramos en la pantalla de gestión de regiones.
Postcondiciones	Las unidades seleccionadas se han movido a la región seleccionada.
Camino Base	
Paso 1	En la pantalla de gestión de regiones, selecciona la región que contiene las unidades a mover.
Paso 2	En el menú de la región, selecciona la opción "Mover", tras lo cual el sistema comprueba que unidades hay en la región.
Paso 3	En el menú de mover, selecciona las unidades que quieres mover.
Paso 4	Elige una de las regiones adyacentes a la seleccionada.
Paso 5	Confirma el movimiento de unidades a la región seleccionada.
Paso 6	Si la región seleccionada es aliada, volvemos al paso 1, si la región seleccionada es enemiga, nos movemos a UC-04.
Excepciones	
Paso 3	El jugador pulsa el botón de cancelar selección.
Paso 4	El sistema nos devuelve a las condiciones previas al caso de uso.

UC-04	**Invasión de región**
Precondiciones	Hemos completado el caso de uso UC-03 con una región bajo control enemigo como objetivo
Postcondiciones	Iniciamos la invasión de una región y entramos en la pantalla de combate.
Camino Base	
Paso 1	Tras realizar el movimiento a una región enemiga, si la región contiene unidades de un oponente, el sistema consulta al jugador si quiere realizar una invasión
Paso 2	El jugador pulsa la opción Si.
Paso 3	El sistema recoge los datos de ejércitos y mapa de la región e inicia un combate.
Excepciones	
Paso 2	El jugador escoge la opción No
Paso 3	El sistema informa al jugador de que el ejército enemigo lo expulsa de la región.
Paso 4	El sistema devuelve a las unidades a su región de origen.
Paso 5	El sistema nos devuelve a la pantalla de gestión de regiones.
Excepcion 3	
Paso 1	Tras realizar el movimiento a la región enemiga, el sistema comprueba que esta no contiene unidades de un oponente.
Paso 2	El sistema avisa de la rendición de la región enemiga al jugador.
Paso 3	El sistema transfiere control de la región al jugador sin iniciar un combate
Paso 4	El sistema nos devuelve a la pantalla de gestión de regiones.

UC-05	**Inicio de combate**
Precondiciones	Hemos entrado en un combate mediante invasión o evento de la historia
Postcondiciones	Las condiciones iniciales del combate han sido configuradas y el jugador ha confirmado las unidades a utilizar.
Camino Base	
Paso 1	El sistema carga el mapa de región y las unidades disponibles del jugador, tras lo que muestra el menú de selección de unidades.
Paso 2	El jugador selecciona cuales de las unidades invasoras participará en el combate.
Paso 3	El jugador confirma su selección de unidades.
Paso 4	El sistema inicia el combate y empieza el turno del bando invasor.
Excepciones	
No aplica	No aplica.

UC-06	**Selección de acción**
Precondiciones	Se ha completado el UC-05 y estamos en el turno del jugador.
Postcondiciones	Se ha finalizado el combate y volvemos a la pantalla de gestión de regiones.
Camino Base	
Paso 1	El jugador selecciona la unidad que desea utilizar.
Paso 2	El jugador selecciona la habilidad que quiere que utilice la unidad y su objetivo.
Paso 3	El sistema aplica los efectos de la acción seleccionada.
Paso 4	Si quedan unidades aliadas que aún no han tomado una acción, volvemos a paso 1, si no, avanzamos.
Paso 5	Si quedan unidades enemigas vivas, el sistema toma control de estas y efectúa sus acciones de turno, tras lo cual vuelve al paso 1. Si no quedan unidades enemigas, avanzamos.
Paso 6	El jugador recibe una notificación de victoria, el sistema le transfiere control de la región y nos devuelve a la pantalla de gestión de regiones.
Excepciones	
Paso 1	Si al jugador no le quedan unidades cuando empieza su turno, es derrotado y se acaba el combate.
Paso 2	El sistema notifica al jugador de la derrota.
Paso 3	El sistema devuelve a las unidades invasoras a su región de origen.
Paso 4	El sistema nos devuelve a la pantalla de gestión de regiones.

UC-07	**Acceso a gestión de unidades.**
Precondiciones	Estamos en la pantalla de gestión de regiones.
Postcondiciones	Entramos en la pantalla de gestión de unidades.
Camino Base	
Paso 1	Desde la pantalla de gestión de regiones, pulsar la tecla a la que se asigna el menú del juego.
Paso 2	En el menú del juego, seleccionar la opción 'Unidades'.
Paso 3	El sistema carga la lista de unidades y nos las muestra por pantalla en el menú de gestión de unidades.
Excepciones	
No aplica	No aplica

UC-08	**Observación de unidades**
Precondiciones	Estamos en la pantalla de gestión de unidades.
Postcondiciones	Entramos en la pantalla de observación de unidades de la unidad seleccionada.
Camino Base	
Paso 1	En la lista de unidades, mover el cursor a la unidad que deseamos observar
Paso 2	El jugador pulsa el botón de aceptación para seleccionar a la unidad a la que apunta el cursor.
Paso 3	El sistema recoge los datos de la unidad y los muestra por pantalla al jugador.
Excepciones	
Paso 1	Si el jugador pulsa la tecla de retroceso, el sistema nos devuelve a la pantalla de gestión de regiones.
Excepción 2	
Paso 3	Si el jugador pulsa la tecla de retroceso en la pantalla de observación de la unidad, volvemos al paso 1.

UC-09	**Cambio de orden**
Precondiciones	Nos encontramos en la pantalla de observación de unidades
Postcondiciones	La unidad está en una nueva posición en la lista de unidades, y nosotros estamos en la pantalla de gestión de unidades
Camino Base	
Paso 1	En la pantalla de observación de unidades, el jugador pulsa la tecla de menú secundario.
Paso 2	En el menú secundario, selecciona la opción Reordenar".
Paso 3	El sistema recoge la lista ordenada de unidades y la muestra por pantalla
Paso 4	El jugador escoge la unidad que tiene la posición deseada de la unidad previamente seleccionada.
Paso 5	El sistema intercambia las posiciones de las dos unidades seleccionadas y nos devuelve a la pantalla de gestión de unidades.
Excepciones	
Pasos 2/4	Si una de las unidades seleccionadas al escoger -eordenar.º en la lista ordenada es nuestra unidad principal, el sistema indica que no puede moverse a la unidad principal y nos devuelve a la pantalla de gestión de unidades.

UC-10	**Modificación de equipo**
Precondiciones	Estamos en la pantalla de observación de la unidad cuyo equipo deseamos modificar.
Postcondiciones	La unidad ha cambiado su viejo equipamiento por el seleccionado.
Camino Base	
Paso 1	En la pantalla de observación de unidades, el jugador pulsa la tecla de menú secundario.
Paso 2	En el menú secundario, selecciona la opción .Equipamiento".
Paso 3	El jugador escoge la ranura de equipamiento que desea modificar.
Paso 4	El sistema recoge la lista de equipamiento disponible para dicha ranura y lo muestra por pantalla.
Paso 5	El jugador selecciona la pieza de equipamiento con la que desea substituir la ya equipada.
Paso 6	El sistema asigna la pieza de equipamiento seleccionada a su ranura y nos devuelve al paso 3.
Excepciones	
Paso 3	El jugador pulsa el botón de retroceso en vez de escoger una ranura.
Paso 4	El sistema nos devuelve a la pantalla de observación de la unidad.
Excepcion 2	
Paso 6	El sistema detecta que la unidad no puede equipar la pieza seleccionada por limitaciones de raza, clase o estadísticas.
Paso 7	El sistema informa al jugador de la limitación.
Paso 8	El sistema nos devuelve al paso 3.

UC-11	Modificación de habilidades
Precondiciones	Estamos en la pantalla de observación de la unidad cuyas habilidades deseamos modificar.
Postcondiciones	La unidad ha cambiado su vieja habilidad por la seleccionada.
Camino Base	
Paso 1	En la pantalla de observación de unidades, el jugador pulsa la tecla de menú secundario.
Paso 2	En el menú secundario, selecciona la opción "Habilidades".
Paso 3	El jugador escoge la ranura de habilidad que desea modificar.
Paso 4	El sistema recoge la lista de habilidades conocidas por la unidad y las muestra por pantalla.
Paso 5	El jugador selecciona la nueva habilidad a equipar.
Paso 6	El sistema desequipa la habilidad de la ranura seleccionada si tiene una asignada, asigna la nueva habilidad a su ranura y nos devuelve al paso 3.
Excepciones	
Paso 3	El jugador pulsa el botón de retroceso en vez de escoger una ranura.
Paso 4	El sistema nos devuelve a la pantalla de observación de la unidad.

UC-12	**Cambio de clase**
Precondiciones	Estamos en la pantalla de observación de la unidad cuya clase queremos modificar.
Postcondiciones	La unidad ha cambiado de clase a la seleccionada.
Camino Base	
Paso 1	En la pantalla de observación de unidades, el jugador pulsa la tecla de menú secundario.
Paso 2	En el menú secundario, selecciona la opción Çlase".
Paso 3	El sistema recoge la lista de clases disponibles para la unidad y las muestra por pantalla.
Paso 4	El jugador selecciona la clase a la que quiere cambiar la unidad.
Paso 5	El sistema asigna la nueva clase a la unidad y nos devuelve a la pantalla de observación de la unidad.
Excepciones	
Paso 3	El jugador pulsa el botón de retroceso en vez de escoger una clase.
Paso 4	El sistema nos devuelve a la pantalla de observación de la unidad.
Excepcion 2	
Paso 5	El sistema detecta que la unidad no puede cambiar a la clase elegida porque no cumple algún requisito para ello.
Paso 6	El sistema informa al jugador de que la unidad no cumple los requisitos de la clase.
Paso 7	El sistema nos devuelve al paso 3.

Capítulo 4

Diseño

Deseño: cómo se realiza o Sistema, a división deste en diferentes compoñentes e a comunicación entre eles. Así mesmo, determinarase o equipamento hardware e software necesario, xustificando a súa elección no caso de que non fora un requisito previo. Debe achegarse a un nivel suficiente de detalle que permita comprender a totalidade da estrutura do produto desenvolvido, utilizando no posible representacións gráficas.

4.1. Modelo de información

Aunque nuestro proyecto no utiliza en sí una base de datos, el modelado relacional de base de datos se adapta muy bien a nuestro modelo de información, y por lo tanto será el modelo que usemos para mostrar las relaciones entre las entidades y funciones de nuestro proyecto.

En la representación del modelo distinguiremos entre las entidades, que representan las distintas entidades del juego para las cuales necesitamos guardar información como se describe en la sección de requisitos de información, y las enumeraciones, que utilizaremos para indicar las relaciones entre entidades que ocurren a través de funciones, como el combate. En una enumeración, si se incluye un atributo de una entidad, es el único relevante, mientras que si se incluye la entidad, necesita todos sus datos.

Para nuestro modelo, las llaves primarias (PK) serán los datos que usamos para identificar cada entidad inequívocamente:

4.2. Módulos del proyecto.

En esta sección describimos los distintos módulos en los que se separa el proyecto y sus relaciones para formar la interacción entre módulos que da lugar a jugabilidad.

4.2.1. Módulos

Gestión de regiones

La gestión de regiones es la parte del videojuego donde distribuimos nuestras unidades en el mapamundi, para defender nuestras regiones y atacar las enemigas, además de desarrollar nuestro territorio.

La gestión de regiones se basa en 3 pilares principales:

Defensa de territorio: La defensa de territorio es el más simple de los 3 pilares de la gestión. Las regiones del mapamundi tienen distintas adyacencias, que indican a que otras regiones se pude viajar desde ellas. Estas adyacencias, por lo tanto, también determinan los caminos de ataque que puede tomar el jugador, y que caminos debe bloquear para poder proteger su territorio. Cuando ocurre una invasión, las unidades que comparten controlador con la región son las asignadas a su defensa, y por lo tanto la estrategia de defensa se basa en la distribución táctica de unidades para bloquear las vías de ataque del oponente.

Invasión de territorio: La invasión del territorio de un oponente se rige de manera similar a la defensa, quedando determinados los caminos posibles a seguir por las adyacencias de regiones. La idea principal de la gestión de regiones es forzar al jugador a encontrar un equilibrio entre la invasión de regiones enemigas y protección de las propias para extender su territorio hasta conquistar todas las regiones del mundo.

Desarrollo del territorio: El desarrollo del territorio es el pilar de la gestión de regiones no basado en combate. Durante la gestión de regiones, además de mover unidades, el jugador puede decidir desarrollar sus regiones mediante la construcción de estructuras, que pueden tener distintos beneficios. Los beneficios que otorga tener controlada una región sin estructuras se limitan a pequeñas cantidades de oro que el jugador podrá utilizar para desarrollar la región. Si el jugador controla todas las regiones que pertenecen a un conjunto (por ejemplo, cada bando empieza controlando el conjunto de sus regiones iniciales, que forman el territorio inicial de su país/reino/clan/etc.) estas también otorgarán un beneficio adicional. Los beneficios que pueden otorgar las regiones y estructuras se dividen en 3 categorías:

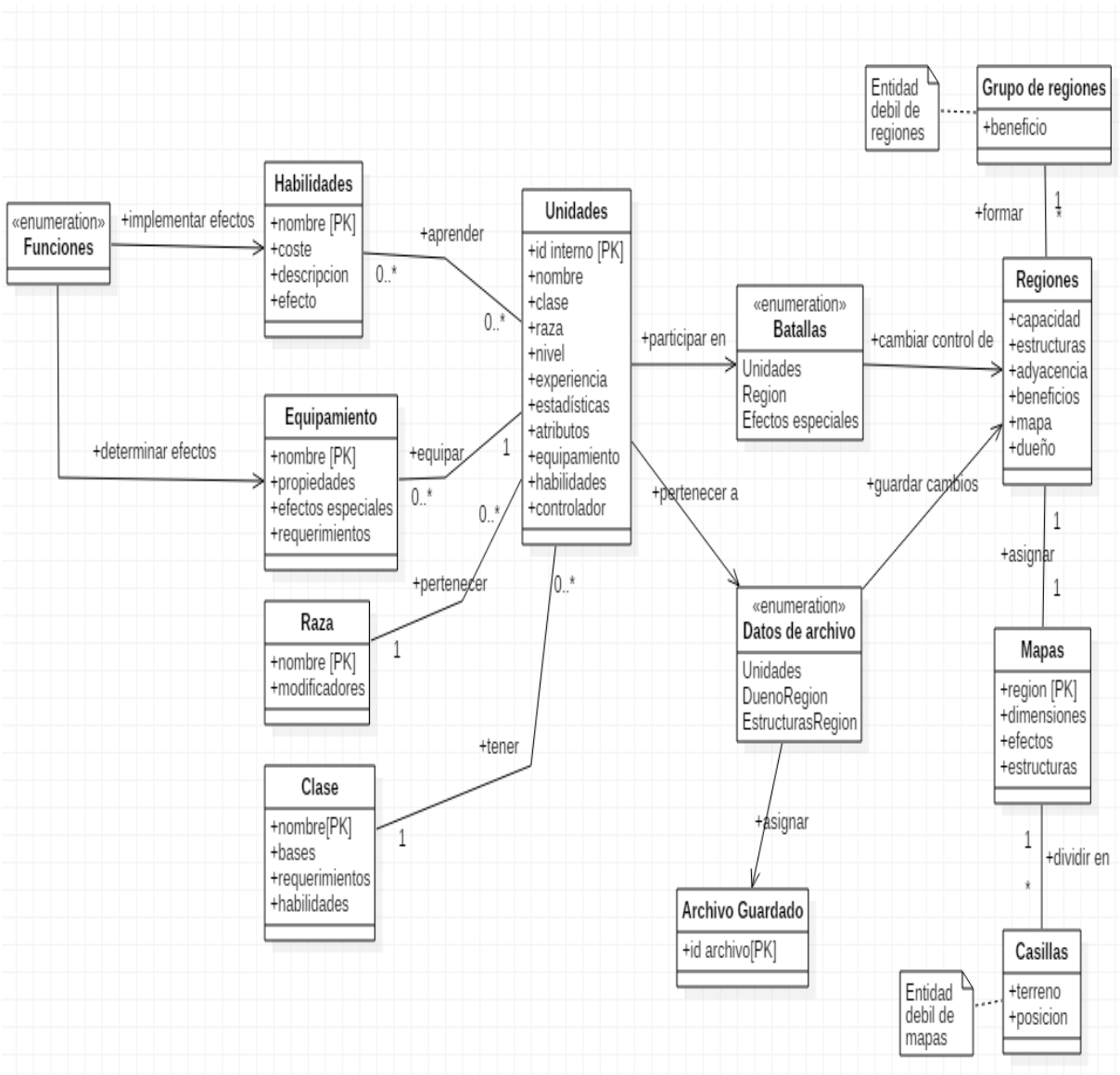

Figura 4.1: Modelo de información

-Económicos: aumentan la cantidad de oro o incluyen objetos en la recompensa mensual. Ejemplos: banco, herrería -Estratégicos: tienen un efecto permanente en el campo de batalla de la región, y a veces también en las regiones adyacentes. Ejemplos: totem divino, torre de maná -Tácticos: tienen un efecto visible con el cual se puede interactuar dentro del campo de batalla de la región en la que se construyen. Ejemplos: murallas, catapultas

Gestión de unidades

En este módulo es donde se incluye la personalización de las unidades del jugador, además del sistema de progresión.

Sistema de progresión: El sistema de progresión del juego se basa en la utilización de unidades en combate, lo cual les hace acumular experiencia, que al llegar a cierta cantidad hace que las unidades suban de nivel, aumentando sus atributos.

Sistema de personalización: El sistema de personalización del juego se basa en la elección de unidades con características innatas (como la raza), y su adaptación al rol de combate que el jugador desea, a través de las clases, equipamiento y habilidades que esta puede adquirir. El sistema de títulos además ayuda a la inmersión del jugador y a su investimento emocional en sus unidades, pues para la obtención de títulos, que proporcionan beneficios específicos, el jugador debe usar las unidades en combate y realizar acciones específicas con ellas, que normalmente requieren cierto esfuerzo.

Relación interna entre los sistemas de gestión de unidades: El módulo de gestión de unidades está diseñado de manera que sus dos sistemas se relacionen para crear una experiencia de jugabilidad interesante, donde las unidades pueden, con el tiempo, adaptarse a las condiciones y estilo de combate con el que practican. Esto se consigue mediante las siguientes relaciones entre los sistemas:

-Estadísticas base: la raza inicial de las unidades determinan sus estadísticas base, lo cual afecta a la progresión de la unidad.

-Tasas de crecimiento: las tasas de crecimiento de las unidades dependen enteramente de la clase que tienen asignada, de manera que si una unidad pasa más tiempo en una clase mágica, su poder mágico y capacidad de maná crecerán mucho más rápido, mientras que una clase centrada en el ataque físico tendrá unas mayores tasas de crecimiento en fuerza y resistencia. De esta manera la personalización de una unidad influye en su progresión, y la progresión de la unidad

abre nuevas oportunidades de crecimiento.

-Aprendizaje de habilidades: las habilidades que puede utilizar una unidad dependen solo de 3 factores, principalmente la clase y nivel, y excepcionalmente, hay títulos que pueden otorgar habilidades adicionales a las unidades, aunque algunas de estas pueden estar restringidas solo a ciertas clases.

-Progresión de clases: debido a que las clases tienen requerimientos que van desde cosas simples, como alcanzar cierto nivel o tener un mínimo de cierta estadística, a más complejas, como la obtención de ciertos títulos o una combinación de estadísticas específica; y a como funcionan las tasas de crecimiento, el sistema de progresión de clases está diseñado para que una unidad que pase más tiempo en ciertas clases tenga mayor facilidad en llegar a la versión superior de estas mismas, ya que requerimientos de altas estadísticas mágicas se alcanzan más fácilmente con tasas de crecimiento de otras clases mágicas, y así para el resto de clases.

Es con estas relaciones que conseguimos crear un sistema de gestión de unidades consistente, pero al mismo tiempo interesante y profundo, debido a la masiva capacidad del jugador para personalizar incluso mínimos detalles de estadísticas de las unidades, lo cual también contribuye al previamente mencionado investimento emocional en las unidades, que se considera una de las partes más importantes para la inmersión del jugador en los RPG.

Módulo de combate

El módulo de combate gobierna el funcionamiento de los enfrentamientos tácticos entre dos ejércitos, y está formado principalmente por los siguientes elementos y por sus correspondientes razones, para generar un combate inmersivo y profundo sin alienar a jugadores novatos:

Mapas de cuadrículas: debido a que la mayoría de RPGs con sistemas de combate por turnos no tienen en cuenta el posicionamiento de las unidades, o se limitan a algo extremadamente básico, como tener una fila delantera y una trasera, los RPGs que consideraban el posicionamiento algo importante y utilizan mapas divididos en casillas de distintas formas se consideraron un nuevo subgénero, al que llamamos TRPG o RPG táctico, refiriéndose a las tácticas utilizadas en enfrentamientos militares. Generalmente, las casillas de los TRPG toman dos formas, hexagonales o cuadradas. Los beneficios de las casillas hexagonales son la mayor cantidad de puntos de contacto de cada casilla, lo cual aumenta la complejidad del campo de batalla y se adapta mejor a grandes cantidades de unidades. Este proyecto utiliza el casillas cuadriculadas debido a que las ventajas de las casillas hexagonales no nos interesan, debido a que nuestra gestión de regiones implica

que a pesar de un gran número de unidades, los enfrentamientos tácticos en mapas ocurren entre pequeños grupos, y no hay buenas razones para aumentar la complejidad mediante casillas hexagonales. Además, las casillas cuadriculadas facilitan también la aplicación de gráficos al terreno, ya que los mapas serán cuadrados y casillas cuadriculadas nos permiten llenar la totalidad del mapa con casillas. Un análisis más profundo de las diferencias entre divisiones de mapas por cuadrículas y hexágonos (y triángulos, aunque en el proyecto no se consideraron) se puede encontrar en el siguiente enlace, el cual fue un artículo muy útil al tomar la decisión. http://www-cs-students.stanford.edu/ amitp/game-programming/grids/ [5]

Efectos de terreno: los efectos del terreno, como aumentos de protección en terreno con cobertura, o daño constante en casillas en llamas, aumentan el impacto del posicionamiento de las unidades, que es una de las características que se consideran centrales al género.

Efectos permanentes: ciertos efectos permanentes durante un combate causados por eventos o habilidades extremadamente poderosas son una buena manera de añadir variiedad estratégica a un combate, obligando al jugador a adaptarse y poniendo a prueba sus dotes estratégicas. Debido a esto, estos efectos suelen mantenerse escasos o inaccesibles durante las partes tempranas del juego, empezando a volverse más comunes a medida que el jugador progresa.

4.2.2. Relaciones intermodulares

Es en esta sección donde explicamos las distintas relaciones entre módulos que convierten el proyecto en un juego de estrategia y táctica completo, donde tanto las decisiones tácticas de combate, como la macro estrategia de gestión económica y de regiones muestran su importancia, y se afectan una a otra.

Relaciones regiones-unidades:
Las relaciones entre regiones y unidades son las más obvias, debido a que la gestión de regiones incluye el movimiento de las unidades en estas regiones, y su utilización para la invasión y defensa de las mismas. Estas relaciones se centran sobre todo en la progresión se unidades a través de la utilización de los beneficios de cada región, y la utilización de las unidades para la protección del territorio controlado.

Relaciones regiones-combates:
Las relaciones entre la gestión de regiones y el combate se basan sobre todo en la gestión de recursos humanos, y son la principal manera de hacer que las decisiones tácticas tengan impacto en las estratégicas y viceversa. Debido a que la gestión de unidades afecta a las unidades que se tienen disponibles en cada combate, y que los combates son la manera principal de protección y expansión

territorial, estos dos sistemas son los más fuertemente enlazados, de manera que se podría tener un TRPG con elementos de simulador de conquista (aunque de mucha peor calidad) sin el sistema de progresión, pero no sin uno de estos dos sistemas.

Relaciones combates-unidades:

Las relaciones entre combates y unidades son las que se centran en los sistemas de recompensa del jugador. Debido a que la personalización y progresión de unidades afectan directamente a las opciones tácticas de cada unidad en combate, y el combate es un paso necesario para la progresión de unidades, pero también para la progresión del juego (ya que es la manera de conquistar nuevas regiones), estos sistemas se enlazan para fortalecer la sensación de progreso del jugador, que notará que sus opciones en combate se abren a medida que sus unidades crecen y se fortalecen, y al mismo tiempo el combate desbloquea nuevas maneras de fortalecer sus unidades a través de la obtención de títulos y experiencia.

Relaciones modulares globales:

Finalmente hay que destacar que las relaciones entre los módulos sirven para formar un videojuego donde todas las facetas de la jugabilidad, ya sea el combate, personalización de unidades, o gestión estratégica de regiones, sean interesantes y profundas de por sí, pero que desbloquean su verdadero potencial de complejidad y diversión cuando se relacionan y afectan unas a otras.

4.3. Diseño de bajo nivel

En esta sección exploraremos los distintos elementos de bajo nivel del proyecto, viendo principalmente los 3 aspectos principales del desarrollo, siendo estos las estructuras de datos, centrales a la creación de unidades y funciones de combate; los sistemas de gráficos, que añaden mucha vida al juego y ayudan a la inmersión del jugador; y el sistema de pila de estados, alrededor del que se basa todo el funcionamiento del proyecto.

4.3.1. Estructuras de datos.

Las estructuras de datos son el aspecto más importante de la manipulación de unidades y combate en el juego. Esta es la parte más simple de programar del proyecto, constando de la creación de estructuras (que en Love2D están basadas en tablas) donde asignamos valores y dependiendo de las funciones que utilicemos modificamos de una manera u otra. Para ver en concreto la implementación, modificación y uso de las tablas que componen nuestras entidades, vamos a analizar parte del código de la aplicación.

```
Character = Class{}

function Character:init(race,level,def)
    self.name = def.name

    self.battleSpriteFront = def.battleSpriteFront
    self.battleSpriteBack = def.battleSpriteBack

    self.baseSTR = def.baseSTR
    self.baseDEX = def.baseDEX
    self.baseENE = def.baseENE
    self.baseRES = def.baseRES
    self.baseTOU = def.baseTOU

    self.STRMOD = race.STRMOD
    self.DEXMOD = race.DEXMOD
    self.ENEMOD = race.ENEMOD
    self.RESMOD = race.RESMOD
    self.TOUMOD = race.TOUMOD

    self.level = level
    self.currentExp = 0
    self.expToLevel = 100

    self:calculateStats()

    self.currentHP = self.HP
end
```

Figura 4.2: Creación de personaje

En la figura 4.2 podemos observar la creación del personaje. Ocultos en la función calculateStats() tenemos también la asignación de movimiento, que para nuestro prototipo se ha fijado en 5 casillas para todos los personajes. La creación del personaje, como se puede ver, es una tarea relativamente sencilla, donde introducimos como argumentos la raza, nivel, y definición de clase (las definiciones de clase es donde guardamos los datos pertinentes a clases, como sus estadísticas base, que son las que usamos aquí), y el código le asigna un sprite y estadísticas base basados en su clase, les aplica modificadores dependiendo de su raza, asigna el nivel que le hemos dado (la experiencia a subir de nivel es 100 constante porque el nivel afecta a la cantidad de experiencia que obtienes), y en calculateStats() obtenemos los stats finales, aplicando a la estadística base su modificador, y si asignamos un nivel mayor de uno, añadiendo estadísticas basándonos en las tasas de crecimiento de la clase asignada. Finalmente asignamos el valor actual de salud al máximo, para que la unidad empiece completamente saludable. (Aquí name es el nombre de la clase, no del personaje)

```
CLASS_DEFS = {
    ['Mago'] = {
        name = 'Mago',
        battleSpriteFront = 'wizard-front',
        battleSpriteBack = 'wizard-back',
        baseSTR = 5,
        baseDEX = 6,
        baseENE = 11,
        baseRES = 8,
        baseTOU = 6,
        STRgrowth = 3,
        DEXgrowth = 4,
        ENEgrowth = 6,
        RESgrowth = 5,
        TOUgrowth = 3,
    },
```

Figura 4.3: Definición de clase

En la figura 4.3 podemos ver una de las definiciones de clase, de donde obtenemos las estadísticas base y tasas de crecimiento de la clase Mago. Aquí es también donde se les asignan los sprites, que dependen de su clase (de manera que todos los magos tendrán el mismo aspecto). En general podemos observar que las estructuras de datos son relativamente simples, y no suelen dar problemas. Ahora vamos a ver su funcionamiento dentro de combate, donde tenemos que manipular estas estructuras dependiendo de habilidades utilizadas.

```
local dmg = math.max(1, attacker.attack*skill.multiplier - defender.defense)

Timer.tween(0.5, {
    [defenderBar] = {value = defender.currentHP - dmg}
})
:finish(function()
    defender.currentHP = defender.currentHP - dmg
    onEnd()
end)
```

Figura 4.4: Fragmento de función de ataque

Aquí podemos observar un fragmento de una función de una habilidad de daño básica, en la que primero calculamos el daño realizado por el ataque, tomando el máximo del ataque físico del atacante por el multiplicador de daño de la habilidad, menos la defensa del oponente. Esto se hace para asegurar que todos los ataques hacen como mínimo 1 de daño (esto se programa así porque 1 daño mínimo no es generalmente suficiente para cambiar el equilibrio de un ataque, pero evita la frustración de gastar un turno sin progresar en absoluto.) Tras esto podemos ver funciones que utilizamos para reflejar los cambios de vida en los gráficos. Tween es una función que nos permite desplazar un objeto o cambiar un valor progresivamente en un tiempo determinado, y lo utilizamos para mover la barra de progreso que sirve para demarcar el porcentaje de vida de los personajes. Tras esto, reducimos el valor de los puntos de vida del defensor en el valor calculado para el daño del ataque.

```
local heal = healer.ENE*skill.multiplier
heal = heal.healCheck(patient)
Timer.tween(0.5, {
    [patientBar] = {value = patient.currentHP + heal}
})
:finish(function()
    patient.currentHP = patient.currentHP - dmg
    onEnd()
end)
```

Figura 4.5: Fragmento de función de cura

Finalmente observamos una función de curación, muy similar a la función de daño, con la diferencia principal de que la curación debe realizar un healCheck, para evitar que el nuevo valor de vida suba por encima del máximo.

4.3.2. Funciones de gráficos

En las funciones de gráficos vamos a observar principalmente 2 cosas, primero, el resultado de una importación de mapa cuadriculado de Tiled al proyecto Love2D, para comprobar como funciona la importación, y como se definen los mapas de combate; y después vamos a ver fragmentos de funciones para demostrar como funciona la animación y asignación de sprites a los personajes.

Mapa de Tiled

Añadimos primero las figuras para que tengan espacio debido a su tamaño:

```
return {
  version = "1.2",
  luaversion = "5.1",
  tiledversion = "1.3.4",
  orientation = "orthogonal",
  renderorder = "right-down",
  width = 16,
  height = 16,
  tilewidth = 16,
  tileheight = 16,
  nextlayerid = 2,
  nextobjectid = 1,
  properties = {},
  tilesets = {
    {
      name = "buch-outdoor",
      firstgid = 1,
      tilewidth = 16,
      tileheight = 16,
      spacing = 0,
      margin = 0,
      columns = 24,
      image = "../../../../Program Files/Tiled/examples/buch-outdoor.png",
      imagewidth = 384,
      imageheight = 192,
      tileoffset = {
        x = 0,
        y = 0
      },
      grid = {
        orientation = "orthogonal",
        width = 16,
        height = 16
      },
      properties = {},
      terrains = {},
      tilecount = 288,
      tiles = {}
    }
  },
```

Figura 4.6: Importación de mapa Tiled

```
  layers = {
    {
      type = "tilelayer",
      id = 1,
      name = "Tile Layer 1",
      x = 0,
      y = 0,
      width = 16,
      height = 16,
      visible = true,
      opacity = 1,
      offsetx = 0,
      offsety = 0,
      properties = {},
      encoding = "lua",
      data = {
        151, 151, 151, 151, 151, 151, 151, 151, 151, 151, 151, 151, 151, 151, 151, 151, 151,
        175, 175, 175, 175, 175, 175, 175, 175, 175, 175, 175, 175, 175, 175, 175, 175, 151,
        199, 199, 199, 199, 199, 199, 199, 199, 199, 199, 199, 199, 199, 199, 199, 199, 151,
        223, 223, 223, 223, 223, 223, 223, 223, 223, 223, 223, 223, 223, 223, 223, 223, 151,
        151, 151, 1, 2, 3, 4, 5, 6, 1, 2, 3, 4, 5, 6, 151, 151,
        175, 175, 25, 26, 27, 28, 29, 30, 25, 8, 9, 80, 81, 50, 5, 6,
        199, 199, 49, 50, 51, 52, 53, 54, 49, 32, 33, 80, 81, 82, 83, 30,
        151, 223, 73, 74, 75, 76, 77, 78, 73, 56, 57, 80, 81, 82, 83, 54,
        175, 151, 97, 98, 99, 100, 101, 102, 97, 80, 81, 104, 105, 106, 107, 78,
        199, 151, 121, 122, 123, 124, 125, 126, 49, 104, 105, 128, 129, 130, 131, 78,
        223, 151, 151, 151, 151, 151, 151, 151, 73, 80, 81, 128, 129, 130, 131, 78,
        223, 175, 175, 175, 175, 175, 175, 151, 97, 104, 105, 128, 129, 130, 131, 102,
        223, 199, 199, 199, 199, 199, 199, 151, 121, 122, 123, 124, 125, 125, 125, 126,
        223, 223, 223, 223, 223, 223, 223, 175, 175, 175, 175, 175, 175, 175, 175, 175,
        223, 199, 199, 199, 199, 223, 223, 199, 199, 199, 199, 199, 199, 199, 199, 199,
        223, 223, 223, 223, 223, 223, 223, 223, 223, 223, 223, 223, 223, 223, 223, 223
      }
    }
  }
}
```

Figura 4.7: Código de cuadrícula de mapa

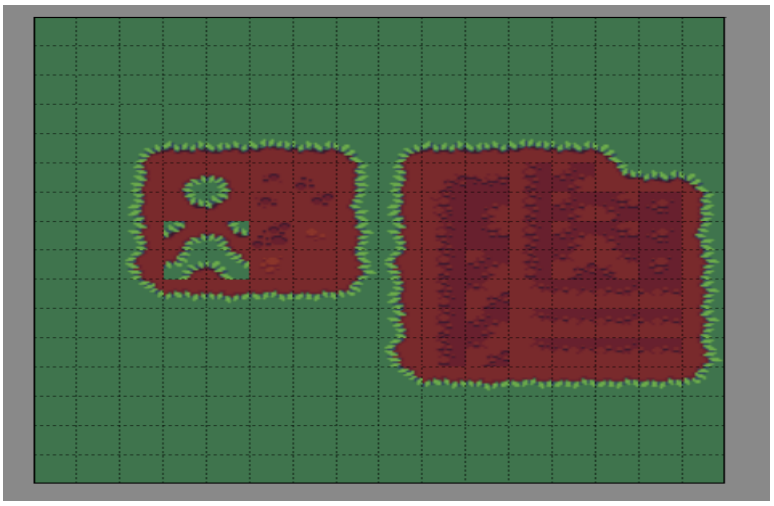

Figura 4.8: Mapa importado [7]

La primera figura (4.6) nos muestra los datos que utilizamos para la importación de mapas de Tiled. Tenemos las versiones de los programas usados, orientación del mapa, orden de renderizado, ancho y alto de las casillas, IDs, y finalmente los sets de casillas. Los sets de casillas, o Tilesets, son imágenes que, divididas en trozos iguales de un tamaño en concreto, se dividen exactamente en un puñado de casillas que nos permite utilizarlas como gráficos, de manera que si asignamos un trozo obtenido de la imagen a una casilla, esta encajará con otras casillas. Como podemos ver en ImageWidth y ImageHeight, ambos números son múltiplos de 16, por lo que al cortarlos en trozos de 16x16 obtenemos las casillas divididas de manera exacta.

Tras esto podemos observar el mapa que hemos utilizado de ejemplo para esta demostración. Los datos del mapa se guardan en forma de matriz, de manera que el código guardado en la posición x,y será el de la imagen utilizada en la misma posición x,y en el mapa. Esto es facil de determinar observando por ejemplo que las 4 filas superiores tienen distintos tonos de verde, y la mayoría de sus códigos coinciden, siendo uno para cada tono de verde, mientras que cuando llegamos a la parte rojiza del mapa, los números empiezan a variar. Por ejemplo, observando la esquina superior izquierda del recuadro rojo izquierdo en la figura 4.8, podemos identificar el código 1 como el de una esquina superior izquierda de un cuadro rojo que entra en contacto con hierba.

Funciones gráficas

Aquí analizaremos dos fragmentos de código. Primero el utilizado para la pantalla del menú inicial, y luego uno utilizado para dibujar y animar los sprites de personaje en un mapa de combate.

```lua
function EntityWalkState:attemptMove()
    self.entity:changeAnimation('walk-' .. tostring(self.entity.direction))

    local toX, toY = self.entity.mapX, self.entity.mapY

    if self.entity.direction == 'left' then
        toX = toX - 1
    elseif self.entity.direction == 'right' then
        toX = toX + 1
    elseif self.entity.direction == 'up' then
        toY = toY - 1
    else
        toY = toY + 1
    end

    -- Si la función nos saca del límite del mapa, cancelamos el movimiento y volvemos a estado Idle
    if toX < 1 or toX > 24 or toY < 1 or toY > 13 then
        self.entity:changeState('idle')
        self.entity:changeAnimation('idle-' .. tostring(self.entity.direction))
        return
    end

    self.entity.mapY = toY
    self.entity.mapX = toX
```

Figura 4.9: Animación de movimiento

```lua
function StartState:render()
    love.graphics.clear(188/255, 188/255, 188/255, 255/255)

    love.graphics.setColor(24, 24, 24, 255)
    love.graphics.setFont(gFonts['large'])
    love.graphics.printf('Fantasia', 0, VIRTUAL_HEIGHT / 2 - 72, VIRTUAL_WIDTH, 'center')
    love.graphics.setFont(gFonts['medium'])
    love.graphics.printf('Presiona Z, Espacio o Intro', 0, VIRTUAL_HEIGHT / 2 + 68, VIRTUAL_WIDTH, 'center')
    love.graphics.setFont(gFonts['small'])

    love.graphics.setColor(45, 184, 45, 124)
    love.graphics.ellipse('fill', VIRTUAL_WIDTH / 2, VIRTUAL_HEIGHT / 2 + 32, 72, 24)

    love.graphics.setColor(255, 255, 255, 255)
    love.graphics.draw(gTextures[self.sprite], self.spriteX, self.spriteY)
end
```

Figura 4.10: Pantalla de inicio

Animación de movimiento

En esta figura (4.9) podemos observar la función preliminar de movimiento de los personajes. Las dos partes importantes para nuestro ejemplo son los dos changeAnimation(). Aquí podemos observar la utilidad de las librerías de Love2D, donde se nos permite cambiar fácilmente la animación de una unidad simplemente con una función y pasándole un argumento. El argumento en este caso es el nombre de la animación a la que queremos pasar, y está hecha de manera que el estado en que estamos determina la primera parte del argumento (si pasamos a idle o walk), mientras que la dirección en la que se ejecuta este movimiento es lo que determina que versión animada de ese estado cogemos, de manera que si por ejemplo pasamos al estado de caminar, y nuestra orientación es a la derecha, el argumento formado será walk-right.

Gráficos de pantalla de inicio

En la figura 4.10 podemos ver nuestro primer ejemplo de pantalla de carga. Lo primero que debemos hacer es ejecutar una limpieza de color de la pantalla con graphics.clear, debido a que llenamos la pantalla de color negro al realizar un fade-in al iniciar el juego. Tras esto imprimimos texto con formato por pantalla, escogiendo primer color y tamaño de la letra que vamos a utilizar, y después imprimiendo el mensaje por pantalla con un simple printf(). Los argumentos del printf aquí indican la posición del texto que vamos a imprimir. Finalmente, para nuestra pantalla de carga deseamos mostrar algunos de nuestros sprites, de maneras similares a las pantallas de carga de juegos como Pokemon, o viejas entregas de la serie Fire Emblem. Para ello, primero dibujamos una elipsis, que servirá para enfocar la atención del jugador, y tras ello hacemos un dibujo de un sprite asignado previamente.

4.3.3. Pila de estados

Finalmente, vamos a hablar de la pila de estados, el pilar alrededor del cual se centra todo nuestro proyecto.

La pila de estados, tal y como indica su nombre, es una pila. Esto significa que organizamos los estados en organización FILO (First In Last Out), es decir, los estados que introducimos primero a la pila serán los últimos en salir de ella, y generalmente serán la base del funcionamiento de los demás.

La idea de la pila de estados fue aprendida y adoptada del excelente curso CS50 de introducción al desarrollo de videojuegos de Harvard [6]. La idea parecía adaptarse perfectamente a un juego RPG, como se ve en el capítulo 7 del curso, donde se desarrolla un clon de Pokemon. Una vez observado el funcionamiento interno de esta técnica, parecía ser incluso más perfecta para un TRPG, ya que la comunicación entre distintas entidades para transferir información de unidades, regiones, mapas, etc podría simplemente realizarse mediante la transferencia de información de estados en la pila, ya que a pesar de que funcione como una pila y solo pueda sacarse de esta el estado más reciente, nosotros tenemos acceso a todos ellos y podemos manipular sus datos como nos sea conveniente.

```
FadeInState = Class{__includes = BaseState}

function FadeInState:init(color, time, onFadeComplete)
    self.r = color.r
    self.g = color.g
    self.b = color.b
    self.opacity = 0
    self.time = time

    Timer.tween(self.time, {
        [self] = {opacity = 255}
    })
    :finish(function()
        gStateStack:pop()
        onFadeComplete()
    end)
end

function FadeInState:render()
    love.graphics.setColor(self.r, self.g, self.b, self.opacity)
    love.graphics.rectangle('fill', 0, 0, VIRTUAL_WIDTH, VIRTUAL_HEIGHT)

    love.graphics.setColor(255, 255, 255, 255)
end
```

Figura 4.11: Estado de FadeIn

La idea de la pila de estados es que podemos guardar la información requerida

para distintas acciones en estados, tras lo cual cuando queremos realizar transiciones entre acciones o pantallas, simplemente hacemos un pop al estado en el que estábamos realizando la acción/viendo la pantalla actual, y empujamos el nuevo estado a la pila. En la figura 4.11 podemos ver un ejemplo de un Fade-InState, que nos permitiría hacer las transiciones entre pantallas con efecto FadeIn, que oscurece la pantalla para la transición de una pantalla a otra, como de gestión de unidades a gestión de regiones.

El funcionamiento del estado FadeIn es el siguiente: cuando empujamos el estado a la pila, su función de inicialización (init) se ejecuta, donde asignamos los colores y el tiempo de transición, tras lo que usamos la función tween, de la que hablamos previamente, para pasar la opacidad de la pantalla con los colores elegidos de 0 al máximo en el tiempo indicado. Debajo podemos ver la función de render(), que simplemente hace que cada vez que llamemos al estado para dibujarlo, este tome los valores de color y opacidad que tiene asignados (la función tween puede aún estar ejecutándose cuando llamamos al estado para renderizarlo, por lo que la opacidad puede ser variable), y llenamos la pantalla con el color indicado, de manera que a medida que aumenta la opacidad, la pantalla va tapándose poco a poco, y debido a que este es el estado de la parte de arriba de la pila, sus gráficos también están encima de los demás, con lo que aunque estemos renderizando por detrás de este una pantalla con objetos, estos estarán tapados por el FadeIn.

En la figura 4.12 podemos observar el código base de la pila de estados. Sus funciones individuales son muy simples debido a que es, en esencia, una pila, pasándose la complejidad al manejo de la pila durante la ejecución del código. Las dos piezas de código más notables son el render, donde podemos observar que tal como se mencionó, se renderizan los estados por orden, lo que resulta que el que esté en la cima de la pila será el que esté encima graficamente, y el push, donde podemos observar que cuando un estado se inserta en la cima de la pila, también entramos en el, no se ejecuta automáticamente.

```lua
StateStack = Class{}

function StateStack:init()
    self.states = {}
end

function StateStack:update(dt)
    self.states[#self.states]:update(dt)
end

function StateStack:render()
    for i, state in ipairs(self.states) do
        state:render()
    end
end

function StateStack:clear()
    self.states = {}
end

function StateStack:push(state)
    table.insert(self.states, state)
    state:enter()
end

function StateStack:pop()
    self.states[#self.states]:exit()
    table.remove(self.states)
end
```

Figura 4.12: Codigo de la pila de estados

Finalmente, vamos a observar el funcionamiento de la pila de estados para la utilización de menús. En la figura 4.13 podemos observar el fragmento inicial del estado del menú de batalla.

```
BattleMenuState = Class{__includes = BaseState}

function BattleMenuState:init(battleState)
    self.battleState = battleState

    self.battleMenu = Menu {
        x = VIRTUAL_WIDTH - 64,
        y = VIRTUAL_HEIGHT - 64,
        width = 64,
        height = 64,
        items = {
            {
                text = 'Skill',
                onSelect = function()
                    gStateStack:pop()
                    gStateStack:push(SkillMenuState(self.battleState))
                end
            },
            {
                text = 'Move',
```

Figura 4.13: Codigo del estado del menú de combate

Para la creación del menú utilizamos la clase panel, donde definimos una selección del menú (siendo cada selección un tipo de datos que conforma un menú completo) como una estructura, en la que tenemos los objetos, o lista de selecciones, la posición en pantalla, marcada con las coordenadas (x,y), y el tamaño de la ventana del menú, en anchura y altura. En nuestro código podemos ver que tenemos un menú en la parte inferior derecha de la pantalla ya que, debido a que Love2D cuenta posiciones empezando desde arriba a la izquierda, incluir la altura y anchura como coordenadas de posición nos colocaría en la esquina inferior derecha, mientras que restarle a estas coordenadas el tamaño de la ventana del menú coloca la esquina inferior derecha del menú en la misma esquina de la pantalla. Tras esto, empezamos a definir los objetos del menú, de manera que les asignamos un texto, que será el texto que se muestra en cada selección del menú.

Dentro de cada selección del menú, la manera más sencilla de implementar nuevas acciones es utilizar el detonador 'onSelect', que nos permite asignar a cada opción código que se ejecutará cuando esa opción sea seleccionada. En nuestro caso, cada opción tiene un pop, que saca el estado de menú de la pila ya que hemos acabado de utilizarlo, y un push(estado), donde el estado que empujamos es el que se corresponde a la acción que hemos tomado. En combate estas podrían ser ejecuciones de habilidades o movimiento, mientras que fuera de combate lo más probable es que sirvan como navegación entre menús, sacando de la pila el estado de menú actual para insertar un estado de menú distinto (en la imagen podemos ver que estamos en el menú de combate, BattleMenuState).

Capítulo 5

Validación

Este capítulo se dividirá en dos partes. Primero veremos el proceso que se llevó a cabo para las pruebas del proyecto, y después haremos un pequeño análisis de que requisitos fueron cumplidos, cuales no, y la justificación de si pensamos que el resultado final ha sido adecuado o no y por que.

5.1. Pruebas del proyecto

Debido a la naturaleza personal del proyecto, y que lo que buscamos principalmente es crear un prototipo funcional de nuestro videojuego, la fase de pruebas del proyecto ha sido ligeramente laxa. A pesar de todo, si buscamos cumplir un mínimo de funcionalidad, y aunque las pruebas no sean muy extensivas, si queremos que sean rigurosas para asegurarnos de que las funcionalidades que buscamos en el prototipo no tienen problemas y se ejecutan correctamente de principio a fin.

Ya que lo que buscamos son pruebas que demuestren una funcionalidad correcta sin centrarse excesivamente en detalles, hemos optado por realizar principalmente pruebas de caja negra, que llevaremos por dos caminos: pruebas de casos de uso, donde mediante la ejecución normal del juego intentaremos comprobar que todos los casos de uso funcionan correctamente, y que no aparezcan problemas simplemente por realizar acciones inesperadas; y pruebas de límites, donde comprobaremos que las funciones del juego no aceptan valores que no deben aceptar. No se realizará un caso de pruebas para el caso de uso UC-01, ya que es de inicio del juego, y si no se cumpliese no indicaría un error sino un proyecto sin función.

Las tablas de pruebas tendrán los siguientes campos:

ID: ID individual que identifica la prueba con el código TST-XX. Los números se asignan por orden de creación y deben ser únicos.

Nombre: El nombre debe describir de manera rápida y poco extensa que se prueba en los casos.
Criterios de inicio: Los criterios de inicio nos indican que parte del programa debe estar completa para poder iniciar el caso de pruebas
Condiciones de inicio: Las condiciones de inicio son condiciones que debemos cumplir antes de iniciar el caso de uso
Categoría: Los tipos de prueba se dividirán en pruebas de casos de uso, funcionales, o de límites
Descripción: La descripción nos dá más detalles que el nombre sobre en que consiste la prueba.
Criterios de aceptación: Los criterios de aceptación son los criterios que debe cumplir la ejecución de un caso de prueba para considerarse aceptado.
Relaciones: Aquí indicamos los casos de uso o requisitos relacionados con la prueba a realizar.

5.1.1. Pruebas de casos de uso

TST-01	**Pruebas de menús**
Descripción	Comprobaremos el funcionamiento correcto de todos los menús del juego.
Categoría	Caso de uso
Criterios de inicio	Esta prueba puede comenzarse cuando todos los menús del juego estén implementados.
Condiciones de inicio	Se ha iniciado el juego correctamente y nos encontramos en la pantalla de gestión de regiones.
Criterios de aceptación	Todos los caminos de los casos de uso relacionados pueden recorrerse correctamente.
Relaciones	UC-02,UC-03,UC-04,UC-06,UC-07,UC-08,UC-09,UC-10,UC-11,UC-12
RESULTADOS	
Resultado:	Los casos de uso se desarrollaron con éxito desde el primer intento.

TST-01	Pruebas de combate
Descripción	En esta prueba comprobamos que las funciones de inicio de combate, movimiento y utilización de habilidades en combate funcionen correctamente.
Categoría	Caso de uso/funcional
Criterios de inicio	Esta prueba puede comenzarse cuando el módulo de combate esté implementado y puedan empezarse combates de cualquier tipo.
Condiciones de inicio	Se ha iniciado el juego correctamente y nos encontramos en la pantalla de inicio de combate.
Criterios de aceptación	Todos los caminos de los casos de uso relacionados pueden recorrerse correctamente. Las unidades de prueba pueden moverse y utilizar todas sus habilidades y estas tienen el efecto esperado.
Relaciones	UC-05,UC-06, UC-07
RESULTADOS	
Resultado:	Los casos de uso se desarrollaron con éxito desde el primer intento. Dos funciones tuvieron errores de funcionamiento en la primera tanda de pruebas debido a utilización de argumentos erróneos, este error fue corregido para la segunda tanda.

5.1.2. Pruebas de límites

Aunque inicialmente se tenía planeada una sección con las pruebas de límites, la restricción de valores que puede insertar el jugador al limitarse su movimiento a menús, y el tipado dinámico de Love2D, que en muchos casos prevee errores (aunque puede causar comportamiento extraño) se combinaron con el foco del proyecto en alcanzar funcionalidad completa y no necesariamente perfecta, y la falta de necesidad de satisfacer las demandas de un cliente, reduciendo en gran medida la importancia percibida de pruebas de límites exhaustivas.

5.2. Análisis de requisitos implementados

En esta sección analizaremos la tasa de completitud de requisitos y nuestra satisfacción con la misma. Para ello incluiremos una tabla simple donde clasificamos los requisitos según han sido completados totalmente, parcialmente, o en ninguna medida.

Requisitos vitales:
Implementación total:RQF-01,RQF-02,RQF-03,RQF-04,RQF-06,RQF-10,RQF-11,RQF-12,RQF-13,RQF-16, RNF-01

Implementación parcial:RQF-05,RNF-05
No implementación:Ninguno

Requisitos importantes:
Implementación total: RQF-14,RNF-02,RNF-03,RNF-04
Implementación parcial:RQF-08
No implementación:Ninguno

Requisitos que quedarían bien:
Implementación total:Ninguno
Implementación parcial:RQF-15,RQF-17
No implementación:RQF-07,RQF-09,RQF-18

Finalmente obtenemos un porcentaje de completitud para cada categoría (contando como medio completos los requisitos parcialmente implementados) de:
Requisitos vitales:92 %
Requisitos importantes:90 %
Requisitos que quedarían bien:20 %

Este se considera un resultado satisfactorio, pues a pesar de que no completamos el 100 % de nuestros requisitos vitales, se considera que estos tienen factores mitigantes, ya que las partes no implementadas fueron de menor importancia. En el caso de RQF-05, gestión de habilidades, no dio tiempo a implementar listas de habilidades dinámicas, de manera que las unidades pudiesen adquirir habilidades nuevas fuera de las permitidas por su clase, esto está relacionado también con haber completado solo parcialmente el requisitos RQF-08, títulos, ya que parte de estos debían poder otorgar habilidades únicas, pero no dio tiempo a incluirlo. Por parte del requisito no funcional RNF-05, este fue completo en el sentido de que podemos distribuir nuestro programa mediante un archivo .zip, pero el desarrollador no quedó satisfecho con tener que recurrir a la versión más simple de la distribución.

Capítulo 6

Conclusiones y posibles ampliaciones

6.1. Conclusiones

Podemos concluir que el proyecto cumplió la mayoría de sus objetivos, y fue un desafío muy interesante realizar un proyecto de esta escala desde 0 mediante aprendizaje completamente autónomo.

El producto final cumple con las expectativas iniciales para un prototipo, y ofrece un buen esqueleto sobre el cual progresar en el desarrollo de un videojuego comercial completo.

La planificación sufrió bastantes mareos debido a la inconsistencia del desarrollador, lo cual fue una de las áreas débiles del proyecto, en la que se debe mejorar sin duda.

Finalmente se añade el comentario de que debido a que parte del problema encontrado con la implementación de requisitos no necesarios fue la gran cantidad de tiempo que consumieron tanto el aprendizaje como la documentación del proyecto, pero que debido a la gran duración del proceso de aprendizaje y a las nuevas herramientas no consideradas inicialmente (como el uso de Tiled), futura implementación de nuevas funcionalidades sea probablemente mucho más rápida.

6.2. Posibles ampliaciones

Las posibles ampliaciones más obvias incluyen la implementación de los requisitos funcionales parcialmente completados o no implementados, además del desarrollo del proyecto hasta alcanzar la estatura de videojuego completo, y la

adición de nuevo contenido al juego, como clases y habilidades adicionales no presentes en el prototipo.

Además de esto, el desarrollador ha considerado distintas funcionalidades adicionales posibles, que incluyen:

-Historia completa que permita viajar por el mundo de Fantasia completando los objetivos de los líderes de las facciones.

-Comercios que ofrezcan nuevas oportunidades de gastar el oro obtenido en objetos.

-Adición de objetos consumibles además de equipo, ofreciendo pseudo-habilidades más débiles y no permanentes a clases que normalmente no podrían acceder a esas capacidades, como pociones de curación para guerreros.

-Adición de asociaciones de mercenarios, donde el jugador pueda contratar nuevas unidades de distintas razas con las que variar su ejército.

-Implementación de razas monstruosas.

Como posible camino para el proyecto también se ve su ampliación para llevarlo a trabajo de fin de máster en un posible máster de diseño, desarrollo y comercialización de videojuegos, aunque esto es ya más difícil de predecir.

Apéndice A

Manuales técnicos

El código será entregado en formato PDF junto al ejecutable, y está adecuadamente comentado función a función para facilitar en todo lo posible su modificación. No se consideró necesario ningún otro manual técnico.

Apéndice B

Manual de usuario

B.1. Instalación

Para la instalación y ejecución del proyecto, simplemente debemos de descomprimir el .zip adjunto y ejecutar el archivo de nombre "Fantasia". Se debe tener cuidado si se mueven los archivos, ya que todos ellos son necesarios y la estructura de carpetas debe mantenerse intacta.

B.2. Utilización del juego

El juego funciona casi en su totalidad mediante menús autodescriptivos, lo cual hace su jugabilidad extremadamente intuitiva. Las teclas asignadas a las distintas funciones del juego son:

-Flechas de dirección: para el movimiento. -Z: aceptar -X: cancelar/retroceso -c: menús secundarios

Bibliografía

[1] Servicio de Gestión de Personal de la USC:Tablas de retribución de personal docente investigador. http://www.usc.es/gl/servizos/sxp/taboas/tabPDI19.html, [En linea; visitada por última vez el 3 de Julio, 2020.]

[2] Información de sueldos medios de ingenieros junior en Glassdoor. https://www.glassdoor.es/Sueldos/ingeniero-junior-sueldo-SRCH_KO0,16.htm [En línea, visitada por última vez el 4 de junio, 2020]

[3] Lista de videojuegos hechos en Love2D, dinámicamente actualizada. https://love2d.org/wiki/Category:Games [En línea, visitada por última vez el 3 de febrero, 2020]

[4] Página principal de Love2D, donde podemos observar los sistemas operativos para los que ofrece el programa. https://love2d.org/ [En línea, visitada por última vez el 22 de febrero, 2020]

[5] Amit Patel, Amits thoughts on grids, *Blog Red Blob Games,* 9 enero 2006, http://www-cs-students.stanford.edu/ãmitp/game-programming/grids/ [Artículo en línea, visitado por última vez el 4 de junio 2020]

[6] Universidad de Harvard, curso CS50, Introduction to Game Development 2018, https://www.youtube.com/watch?v=GfwpRU0cT10 [Serie de vídeos completamente disponibles en youtube, visitado por última vez el 27 de junio, 2020]

[7] Gráficos de mapas y tilesets utilizados en Tiled proporcionados en el instalable para su utilización como ejemplos.